From Traces to Guarded Programs: Evidence-Gated Compilation of Recurrent Agent Workflows

Anonymous authors
Paper under double-blind review

Abstract

Tool-using agents often call a language model at every step, even when they repeat the same read-only lookups. Replacing that repeated reasoning with a fixed program could cut cost, but repetition alone does not make it safe: a tool argument may depend on an earlier result, a "read-only" tool may have side effects, and replaying the same calls may still change the final answer.

We introduce *guarded agentic compaction* (GAC), which compiles the recurring read-only opening of an agent's workflow into a deterministic program only when the evidence justifies it, and otherwise leaves the agent unchanged. GAC grounds tool arguments in inputs, prior results, or observed invariant literals, treats writes, approvals, handoffs, and unknown effects as hard stops, checks inputs and outputs at run time, and deploys a program only after enough independent examples pass a statistical test.

On three real GitHub workflows, the compiled agent had similar observed accuracy to the original agent (90 of 90 held-out tasks correct, versus 89 of 90) and never failed a task the original agent solved (one such failure in 810 held-out GitHub episodes in six specified studies), while making about two-thirds fewer model calls, finishing in about a third of the time, and costing less than half as much per task. On 132 fresh pull requests created after our data snapshot, it made no end-to-end errors, which bounds its error rate below 2% with 90% confidence for open and merged pull requests, assuming independent, identically distributed records. On BIRD, a public text-to-SQL benchmark, a live schema-first agent's opening was compiled on 26 of 31 databases and on a second model family, cutting model calls by 45–50% with no detected accuracy loss; agents choosing tables per question were refused. Two public benchmarks show no measured benefit: NESTFUL holds at most 26 examples of any workflow, against the 92 required, and on AppWorld the one admitted program fits almost no ReAct or plan-and-execute run. With a second model family and a second provider, GAC rediscovers the same programs on two of the three GitHub workflows and declines the third. We also report its limits: its main guarantee mostly confirms that enough examples were seen, the savings hold within one data snapshot, and its earlier corruption tests showed no protection. A separate live study on 72 issue groups with constructed requests finds that request-scope checks improve ordinary-response task success by 8.8 percentage points over their removal (descriptive 95% interval 3.7–13.9), while completeness checks show no clear benefit when the agent can recover missing evidence.

GAC reframes agent optimization as evidence-gated specialization: compile only what is justified, and keep the model's reasoning everywhere else.

1 Introduction

The standard agent loop pays for a model decision at every tool boundary (Yao et al., 2023). That flexibility is valuable while a workflow is novel. When an agent repeatedly takes the same read-only evidence path with invariant data dependencies, those decisions offer a candidate for specialization; their prevalence across deployed agents remains unmeasured.

Removing that decision is not ordinary caching: a tool argument may depend on an earlier observation, a repeated sequence may cross an approval or write, a read-only tool may hide an effect, and

replaying the same outputs may still change the answer. The question is when a recurrent model-mediated region may be replaced by a deterministic program, and when it must be refused.

Consider held-out issue #4420. Given only `issue_number=4420`, the observed trace calls `record(4420)`, passes the *returned* `record.issue_number` to `labels` and `comments`, and invokes the latter with `limit=100`. All 132 discovery executions take that three-read route (130 with `limit=100`, two with `limit=1`), so a recurrence-only optimizer would emit all three reads. GAC instead retires the *region*: `comments.limit` is not an invariant literal and is reachable neither from the entry state nor from either earlier result, so one unwitnessed slot blocks the three-read candidate as `ungroundable_slot`. What survives is the maximal justified prefix `record` → `labels`; the agent still chooses the limit and writes the answer. That two-read artifact passes 92/92 calibration groups at $\alpha = 0.05$ (upper bound 0.0498).

An earlier 18-record study supplies the counterexample: its three-read artifact, admitted at a looser $\alpha = 0.10$ (a 45-group floor), passed all 45 calibration replays and the tool contract on every held-out record, yet issue #6602 lost the URL from a Markdown link and only 17/18 downstream answers stayed exact; a provider-free continuation check, run after the model's answer, detects that miss. Together the two cohorts show why recurrence only proposes a candidate (Fig. 2 draws the first).

The closest precedent is trace compilation: Dynamo (Bala et al., 2000) and tracing JITs (Gal et al., 2009) compile a hot path behind *guards* and fall back when one fails, as deoptimization (Hölzle et al., 1992) restores unoptimized execution. Agent traces admit the same structure (family mining, an entry contract, fallback to the unchanged agent) but compile only *initial* read-only prefixes (Eq. (2)) and add three obligations: tool arguments must be shown *grounded* in observable state; declared *effects* must permit speculation, since the "interpreter" calls the outside world; and the residual error rate, being statistical, needs a finite-sample bound.

We study these obligations through *guarded agentic compaction* (GAC), a compile-or-retire trace-to-program system for recurrent read-only prefixes (Fig. 1). The core contribution is not "agent compilation" in general, but an admissibility argument for trace-derived specialization:

1. A guarded specialization pipeline (Alg. 2) combining typed provenance, effect and position barriers, bounded synthesis, runtime verification, and finite-sample compile-or-retire admission (Alg. 1, Proposition 1) with the certificate level and its event stated per artifact. The bound is a standard exact binomial interval under a union bound over a fixed grid; the contribution is the admission rule and its composition with trace-derived agent compilation. Every stage can refuse, and the default output is the unchanged agent.
2. A real-provider evaluation across three public-record workflow families; a cross-repository extension on newer records with admissions and principled retirements; a post-cutoff cohort with an end-to-end certificate; a live SQL agent on the public BIRD benchmark whose fixed schema-reading opening compiles on 26 of 31 databases (Section 5.3); three pre-registered drift ablations (two uninformative by construction, one adverse); replications on a second model family and provider; and a live safeguard ablation separating request-scope protection from recoverable missing evidence (Section 5.5).
3. Calibrated refusal on public trace benchmarks (Section 5.4): NESTFUL, the largest trace corpus the compiler ingests here, clears provenance, synthesis, and replay with no wrong execution yet retires below the exact sample floor (API-Bank and BFCL v4 likewise), a corpus-size fact that bounds what such benchmarks can certify; AppWorld admits a region that is eligible on 2,339/2,340 released full-code trajectories but 1/4,680 ReAct or plan-and-execute ones, separating *admissible* from *dispatchable*.

2 Problem Formulation

Terminology. An *episode* is one recorded run of the agent; a *group* is the set of episodes that share an independence unit (one record, one scenario); a *region* is a contiguous span of one episode's tool calls; a *family* is the set of regions across episodes that share a canonical shape; a *candidate* is a family the compiler attempts and, once synthesized, its thresholdless tuple (P, H, V, q, M); a *program* is the deterministic code synthesized for it; an *artifact* is a program together with its guard, verifier, score, threshold, and manifest pins; and a *prefix* is a region that starts at the first model boundary. Compilation is the offline act; compaction is the runtime substitution it licenses.

Episodes and candidate regions. An episode $E = (z, M, e_{1:T}, y)$ contains an entry-state snapshot z, a content-addressed execution manifest M, ordered events e_t (model requests and responses, tool calls and results, handoffs, guardrails, approvals, errors, commit boundaries), and an observable outcome y; c_j denotes the j-th tool call and o_j its result. Each tool f carries an application-owned effect declaration $\epsilon(f)$ and capability flags such as *speculatable*; an unknown effect is not a read.

A candidate region $R = [a, b]$ begins at a model-request boundary and ends after a tool result. It is *groundable* when every call argument is either a literal invariant across the supporting traces or an expression over entry state or prior results inside R,

$$\forall c_j \in R,\ \forall u \in \mathrm{args}(c_j): \quad \mathrm{Lit}(c_j, u) \ \vee\ \exists\, s \in \mathrm{Src}(z, o_{<j}),\ \exists\, g_{j,u} \in \mathcal{L}:\ u = g_{j,u}(s), \tag{1}$$

where $\mathrm{Lit}(c_j, u)$ holds when the slot carries one literal invariant across every supporting trace (the implemented rule; a slot the schema declares literal-only may be bound in no other way), $\mathcal{L}$ is a closed, bounded transform library, and $\mathrm{Src}(z, o_{<j})$ enumerates the flattened paths of z and of each prior result o_i, $i < j$; and *effect-admissible* when every tool in R is declared read-only, speculatable, replayable, and inside one principal and isolation partition. The witness is deliberately *single-source* — one chain over one observation, not a join over the history — which is the quantifier Alg. 3 (Appendix B) implements. Handoffs, approvals, writes, errors, and unknown effects are hard barriers. Because the deployed runtime resolves artifacts at the initial model boundary, the compiler also imposes a *position invariant*:

$$a = 0 \quad \text{in the normalized model-boundary index.} \tag{2}$$

A suffix program dispatched at entry can reorder or duplicate calls the agent has already made; a pilot that violated Eq. (2) produced exactly that fault (Appendix E). Everything that follows is therefore an *initial read-only prefix* compiler; the AppWorld result of Section 5.4 prices what that costs.

Selective optimization objective. The compiler emits an artifact $A = (P, H, V, q, \eta, M)$: a deterministic program, hard guard, output verifier, nonconformity score, threshold, and manifest pins. Let $x = (z, M', \omega)$ be a future entry state, runtime manifest, and external state, where $M' \simeq M$ is equality on every pinned compatibility field and ω affects outcomes and cost but is unobservable at the dispatch boundary. Dispatch is *selective*,

$$d_A(x) = \mathbf{1}\{\, H(z, M') = 1 \ \wedge\ q(z) \le \eta \ \wedge\ M' \simeq M \,\}, \tag{3}$$

and any failed precondition runs the unchanged baseline B. If execution fails before a commit and staging proves the attempt clean, B runs; a failure whose effects cannot be proved clean is an incident, not a fallback. Fix $L(A, x) = 1$ for an externally visible task/contract violation or incident after dispatch, $L = 0$ for a clean abort followed by unchanged B, and let $C(A, x)$ be the complete realized cost of the attempt including a clean fallback. For a future group $G = (x_1, \ldots, x_m)$, set $D_A(G) = \max_i d_A(x_i)$ and $W_A(G) = \max_i d_A(x_i) L(A, x_i)$. We seek

$$\max_A\ \mathbb{E}_{G \sim \mathcal{D}_{\mathrm{grp}}}\Big[\, \textstyle\sum_{x \in G} d_A(x)\,\{C(B, x) - C(A, x)\}\,\Big] \quad \text{s.t.} \quad \Pr_{G \sim \mathcal{D}_{\mathrm{grp}}}[W_A(G) = 1 \mid D_A(G) = 1] \le \alpha. \tag{4}$$

Both halves take the same group measure. *First*, the constraint conditions on dispatch: pre-dispatch abstention does not enter its denominator, although it earns no savings in the objective; by convention its conditional risk is zero when population dispatch probability is zero. *Second*, returning *no* artifact is feasible, so RETIRE is a legitimate output, and Eq. (4) is a design target rather than a solved program — the implementation calibrates a bounded ranked set and retains nondominated survivors, and we prove neither optimality nor any guarantee for that search beyond the per-candidate and fixed-m cases of Section 3. *Third*, every certificate in Section 5 except one end-to-end certificate (Section 5.1) instantiates $L(A, x) = 1$ as a *replay-contract violation* of the compiled region (Appendix C), so the continuation is measured, not certified. The denominator counts groups with at least one attempt admitted by the hard guard and score. A clean interpreter or verifier failure after admission has $L = 0$ and remains in that count.

3 Guarded Agentic Compaction

Figure 1 gives the six stages; Alg. 2 (Appendix B) lists them in full. The order is part of the argument: each stage can only weaken a candidate, so a later stage may reject one an earlier stage passed, but never authorize one an earlier stage blocked.

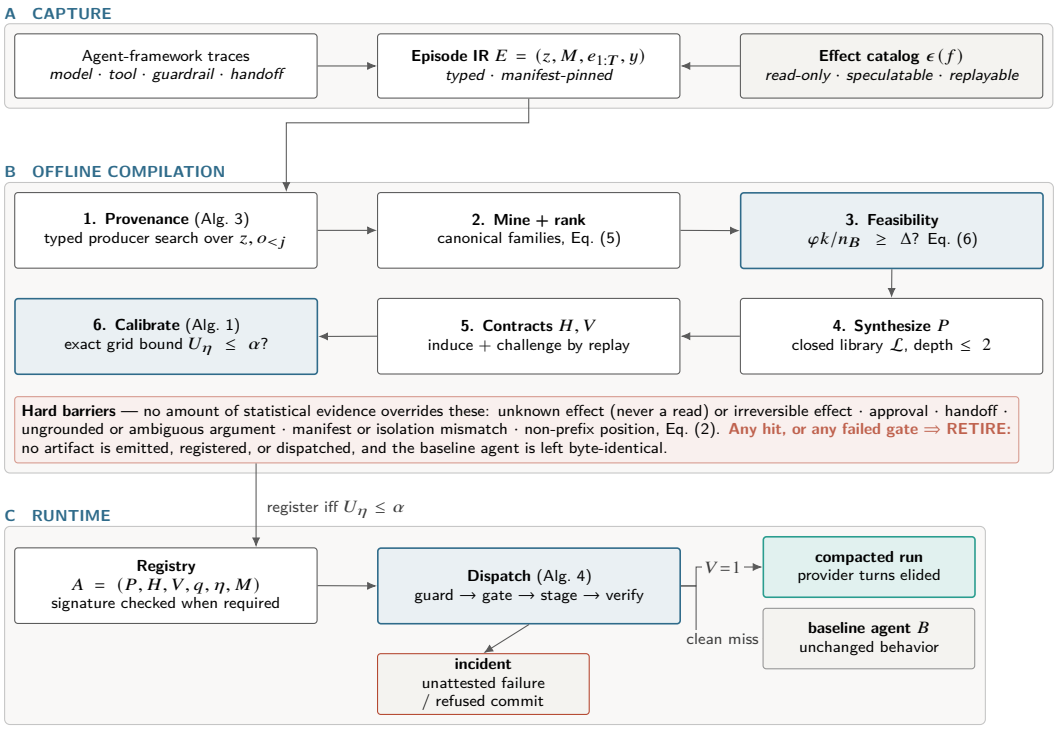


Figure 1: Architecture of GAC. **(A)** Capture normalizes traces into one typed Episode IR against a declared effect catalog. **(B)** Compilation is a cascade of independent rejection points; the shaded band lists barriers no statistical evidence can override. **(C)** At runtime one terminal edge compacts; clean rejections return the unchanged agent, and unattested failures raise an incident.

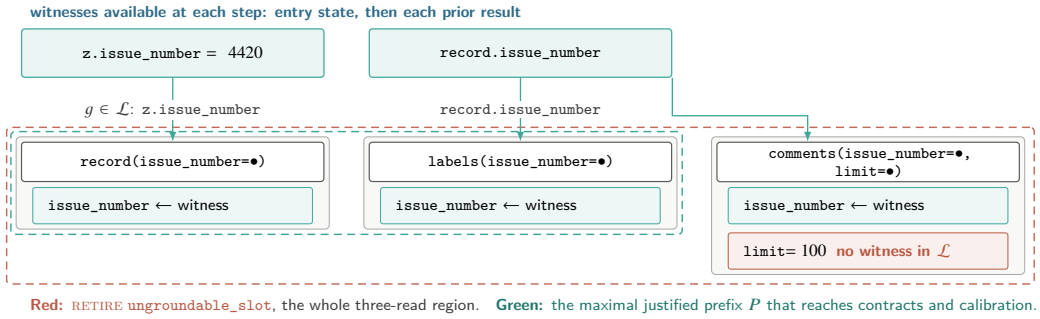


Figure 2: Equation (1) on the held-out issue-4420 record: both `issue_number` slots have an expression in $\mathcal{L}$ over the entry state or a prior result; the `limit` literal varies across the supporting traces (100 in 130 of 132, 1 in two) and has none, so the three-read region retires and only the two-read prefix is emitted.

1. Typed provenance. A capture adapter normalizes framework traces into the Episode IR; manifests pin prompt, policy, guardrail, tool, model, SDK, tracer, entry-contract, and effect-catalog identities, and episodes with different compatibility keys are partitioned before any learning. For each call-argument slot, the program-argument trace graph (PATG, Alg. 3) searches entry-state and prior-result paths for typed values reachable by a bounded transform (database-style provenance (Cheney et al., 2009) for tool arguments). Two slot marks make the region miner block rather than guess: a slot with *no* witness is unresolved in the bounded library, and one with more than $\kappa = 3$ witnesses is

ambiguous (Fig. 2); either retires the whole region. A repeated tool name or a successful replay is no witness; an invariant literal is (Eq. (1); the `limit=3` slots of the PR-outcome and backlog artifacts).

2. Region mining and ranking. The miner enumerates model-boundary-to-result windows of at most $w_{\max}$ tool calls, qualifies live-ins and effects against a human-signed rather than inferred catalog, and hashes tool signatures, dependency topology, run shape, and live-in/live-out shape while abstracting literals. Families require support across independent scenario *groups*, not raw event frequency. Survivors are ranked by an MDL-inspired trade-off that pays for expected savings and charges for heterogeneity, effect exposure, and program size:

$$\text{score}(F) = \underbrace{s_F\, \bar{k}_F\, c_m}_{\text{support-weighted saving}} - \lambda_1 \mathsf{Ent}(F) - \lambda_2 \rho_{\text{eff}}(F) - \lambda_3 |F|, \tag{5}$$

with s_F supporting train groups, $\bar{k}_F$ the mean removable model requests, c_m their unit cost, and $\mathsf{Ent}(F)$ the Shannon entropy over canonical variants inside the family. Equation (5) affects only *which* family is tried first.

3. Feasibility ceiling. Before any synthesis runs, an estimator bounds what the compiler could achieve with a perfect verifier and a gate that never abstains:

$$\Delta_{\max} = \frac{\varphi\, k}{n_B} \;\geq\; \Delta \quad \text{(pre-synthesis filter)}, \qquad \varphi\, \rho\, k \;\geq\; \Delta\, n_B \quad \text{(realized, post-verification)}, \tag{6}$$

with n_B baseline model requests per episode, φ the fraction of episodes holding an eligible region, k the requests removed per successful dispatch, and ρ the verifier pass rate. Stage 3 evaluates only the left form ($\rho = 1$). This ceiling concerns requests removed by the prefix at fixed continuation workload; changes in downstream reasoning or fallback can change total requests. A workload below this structural target is declined.

4–5. Bounded synthesis and contracts. Synthesis searches the 23-operator library of Table 6 at depth at most two; bindings are ranked by cross-group stability, and a group-wise refit rejects bindings exploiting row-level coincidence. The hypothesis space is small enough to read, in the spirit of programming-by-example (Gulwani, 2011) and bounded sketching (Solar-Lezama et al., 2006), and never generates arbitrary Python. Hard guards then constrain entry types, required fields, categorical sets, numeric intervals, patterns, isolation keys, allowed effects, and manifest pins; verifiers constrain live-out presence, type, cardinality, empirical hulls, provenance, effect multisets, and call counts. These are candidate invariants, not proofs — which is why a statistical gate follows rather than replaces them — and under tool-layer corruption their measured protective value was null or adverse (Section 7).

6. Exact risk-gated admission. Algorithm 1 decides deployment. The score q is a logistic model over entry-observable features — missing fields, hull margin, temporal drift, and provenance ambiguity — fitted by regularized maximum likelihood on *dev* groups only, never on calibration groups, then frozen together with the finite threshold grid $\Lambda = \{0.02, 0.05, 0.08, 0.11, 0.14, 0.17, 0.20, 0.25, 0.30, 0.40, 0.50\}$. q is *fitted* to predict an *unproductive* dev group — guard rejection, abstention, or manifest mismatch — never a wrong execution, since ReplayAndChallenge already retired those; the bound below is *counted* only on dispatched groups that turned out wrong (Appendix C). For each $\eta \in \Lambda$ the compiler counts admitted calibration groups (n_η) and those containing at least one post-dispatch violation (k_η), and inverts the one-sided exact binomial test (Clopper & Pearson, 1934) at a Bonferroni-corrected level, with explicit empty and all-violation cases:

$$U_\eta = \begin{cases} 1, & n_\eta = 0 \text{ or } k_\eta = n_\eta, \\ \text{Beta}^{-1}\!\left(1 - \frac{\delta}{|\Lambda|}; k_\eta + 1, n_\eta - k_\eta\right), & 0 \leq k_\eta < n_\eta, \end{cases} \qquad U_\eta\big|_{k_\eta=0} = 1 - \left(\frac{\delta}{|\Lambda|}\right)^{1/n_\eta}. \tag{7}$$

It admits the largest-coverage threshold with $U_\eta \leq \alpha$ and retires otherwise — a fixed-grid instance of learn-then-test (Angelopoulos et al., 2025). The closed form on the right is the operative constraint: at the registered $\alpha = 0.05$, $\delta = 0.1$, $|\Lambda| = 11$ (Appendix C), a zero-violation family still needs $n_\eta \geq 92$ admitted groups, and the default output is refusal.

Algorithm 1 CALIBRATE — fixed-grid exact selective admission. The score q and the grid Λ are frozen *before* this procedure sees a calibration group, which is what makes the union bound legitimate.

Input: frozen candidate $A_0 = (P, H, V, q, M)$ without threshold; calibration groups $\mathcal{K}$ with fixed replay-contract labels $L(A_0, x)$; grid Λ; budgets α, δ
Output: threshold η with a simultaneous guarantee, or $\perp$ (RETIRE)

$\Pi \leftarrow \emptyset$ ▷ admissible (coverage, threshold) pairs, distinct from Alg. 2's $\mathcal{A}$
for all $\eta \in \Lambda$ **do**
  define $d^\eta(x)$ by Eq. (3) with threshold η; $D^\eta(g) \leftarrow \max_{x \in g} d^\eta(x)$; $W^\eta(g) \leftarrow \max_{x \in g} d^\eta(x) L(A_0, x)$
  $K_\eta \leftarrow \{\, g \in \mathcal{K} : D^\eta(g) = 1 \,\}$; $n_\eta \leftarrow |K_\eta|$; **if** $n_\eta = 0$ **then continue** ▷ vacuous bound $U_\eta = 1$
  $k_\eta \leftarrow |\{\, g \in K_\eta : W^\eta(g) = 1 \,\}|$ ▷ *wrong after dispatch* only, never abstention
  $U_\eta \leftarrow 1$ if $k_\eta = n_\eta$, else $\mathrm{Beta}^{-1}\big(1 - \frac{\delta}{|\Lambda|};\, k_\eta + 1,\, n_\eta - k_\eta\big)$; **if** $U_\eta \le \alpha$ **then** $\Pi \leftarrow \Pi \cup \{(n_\eta/|\mathcal{K}|, \eta)\}$ ▷ Eq. (7)
end for
return $\perp$ if $\Pi = \emptyset$, else the η component of $\mathrm{lexmax}_{(\mathrm{cov}, \eta) \in \Pi}(\mathrm{cov}, \eta)$ ▷ largest empirical group coverage; larger threshold breaks ties

That floor also bounds what selectivity is *certifiable*: coverage $c = n_\eta/|\mathcal{K}|$ needs $|\mathcal{K}| \ge \lceil 92/c \rceil$, so on the $|\mathcal{K}| = 92$ primary cohorts — the smallest satisfiable pool — the only admissible coverages are $c \in \{0, 1\}$ whatever q ranks, since abstaining on one group leaves $U_\eta = 0.0503 > \alpha$; intermediate coverage can be *observed* but not certified (Appendix C, Fig. 4). Nor does the event itself have support on the GitHub cohorts: the compiled steps' argument pattern is invariant across all discovery traces per family and labels replay those calls over a pinned snapshot. Those certificates are therefore support-sufficiency statements at full coverage ($n_\eta \ge 92$; $\delta = 0.1$ is the budget at which that floor is satisfiable, $\delta = 0.05$ needs 106) that cannot tell whether q ranks dispatch risk.

Proposition 1 (Per-candidate selective-risk admission)**.** *Condition on train/development data and fix one candidate (P, H, V, q), group-level violation rule, and finite grid Λ before observing calibration. If calibration groups are i.i.d. from the future-group distribution, then with probability at least $1 - \delta$, every $\eta \in \Lambda$ satisfies $r_\eta \le U_\eta$, where r_η is $\Pr(W^\eta = 1 \mid D^\eta = 1)$ when population admission is positive and is defined as zero otherwise. Consequently the selected $\hat{\eta}$ satisfies $r_{\hat{\eta}} \le \alpha$ whenever $U_{\hat{\eta}} \le \alpha$.*

Appendix A gives the proof. The guarantee is per fixed candidate and conditional on i.i.d. groups, which the cohorts assume rather than establish (Section 7). Conditional on train and development data the number m of candidates reaching calibration is fixed, so a run is compiler-wide when $m = 1$ (Corollary 2) or when the per-threshold budget $\gamma = \delta/|\Lambda|$ of Eq. (7) is divided by m; Appendix C reports m and both bounds for every artifact.

4 EXPERIMENTAL SETUP

GitHub workflows. Every sample begins with one issue or pull-request number from a pinned public snapshot (helmo, 2025); the agent must make read-only calls and return exact, source-grounded fields. The three primary families use 132 discovery records each, from whose exact traces a stable hash draws 16 train, 8 dev, and 92 calibration groups ($\mathcal{K}$; the rest stay unused), and 30 balanced held-out records, with live provider calls, each pairing a read-only evidence path with an exact answer contract (a category, a title, and a verbatim excerpt; Table 13, Appendix E). The exact-trace filter retains 130–132 of 132 records but still conditions the certificate on selection; Appendix G details this limitation.

Cross-repository extension on newer records. A narrower two-read PR-outcome task spans five frozen public repositories. Held-out records are newer than discovery records but inside the same frozen snapshot, which every artifact pins; selection is sealed before provider calls, and repositories without an admitted artifact stay in the result. A balanced rerun on three repositories controls the class mix.

Public benchmarks. On BIRD (Li et al., 2023a) a live SQL agent runs over pinned read-only databases (Appendix F.4). NESTFUL (Basu et al., 2025) and AppWorld's gold solutions (Trivedi et al., 2024) supply executable traces, plus released AppWorld agent runs; no model runs on these two, so no efficiency claim. API-Bank and BFCL v4 (Li et al., 2023b; Patil et al., 2025) behave like NESTFUL (Appendix F.3).

Table 1: Primary live-provider results ($n = 30$ held-out records per family). "Interfaces" counts provider-visible tool interfaces. All conditions share records, model, and grader; *Manual* is hand-written with full workflow knowledge (Table 11, Appendix D).

	Exact held-out contract			Reduction of compiled vs. baseline (%)				
Family	Base	Compiled	Manual	Requests	Interfaces	Tokens	Latency	Cost
Issue-type routing	30/30	**30/30**	30/30	50.0	0.0	39.5	51.7	32.0
PR-outcome audit	30/30	**30/30**	30/30	75.0	66.7	80.8	73.0	75.3
Backlog-attention routing	29/30	**30/30**	30/30	74.8	66.3	81.4	68.9	75.1
Weighted total ($n = 90$)	89/90	**90/90**	90/90	66.6	44.2	63.1	64.2	58.7

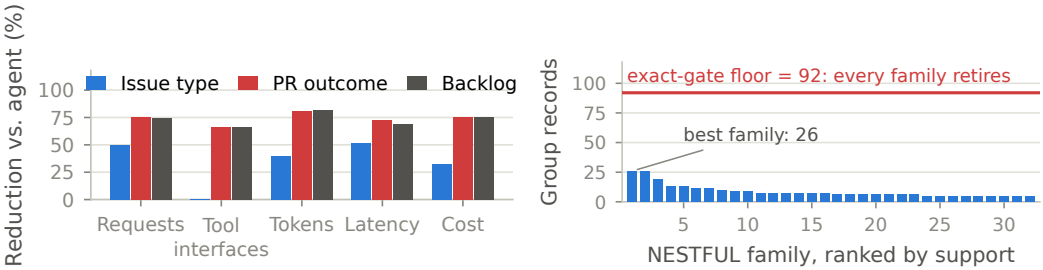


Figure 3: Left: reduction against the unchanged agent per family (30 held-out records each; the values of Table 1). Right: NESTFUL's 32 recurring families ranked by group-record support against the 92-group floor of Eq. (7); the best reaches 26, so every family retires (Section 5.4).

Provider configuration. The primary studies use OpenAI `gpt-5.6-luna` via the `openai-agents` Responses interface (OpenAI, 2026a) at reasoning effort `low`, priced at list rates (OpenAI, 2026b); the replications use `gpt-6-luna` and Anthropic `claude-sonnet-5` under the same settings (Appendix C). Conditions within a family differ in structure alone.

Conditions, metrics, and statistics. Each primary GitHub family compares the unchanged baseline B, the compiled artifact, and a hand-written pre-model program on the same records, so comparisons are paired. Quality is exact task-contract satisfaction; resources are requests, interfaces, tokens, latency, and estimated cost; tests are paired exact McNemar (McNemar, 1947), bootstrap intervals (Efron & Tibshirani, 1993), and Wilcoxon signed-rank (Wilcoxon, 1945), with gate multiplicity controlled by the Bonferroni term of Eq. (7).

5 Results

5.1 Primary workflows: preserved contracts, less work

Across the 90 held-out records, GAC satisfies 90/90 exact contracts and the unchanged agent 89/90 (Table 1, Fig. 3 left), reducing provider requests by 66.6%, tokens by 63.1%, latency by 64.2%, and cost by 58.7% (per-record values and bootstrap intervals in Tables 7 and 10, Appendix D); issue-type routing, a two-read prefix, is the low end of every range in Fig. 3. These are marginal per-episode savings within one snapshot: every artifact pins its snapshot digest and abstains on any other (a new snapshot is recompiled), and discovery cost alone breaks even after 411, 183, and 181 episodes per family (Appendix D).

The discordant pair is one baseline-only miss (backlog record 5189, an ungrounded excerpt) against zero compiled-only misses (McNemar $p = 1$; one-sided 95% bound on compiled-only discordance 3.3%); a rerun of the 30 records scored 30/30 in every arm (Appendix G), so no accuracy difference is detected; equivalence and noninferiority were not established. Programs hand-written with full workflow knowledge also reach 90/90, tie on requests, use fewer interfaces and tokens, cost marginally less, and lose only on latency (Table 11): we claim discovery and admission, not runtime dominance.

Table 2: The two trace benchmarks with no measured GAC benefit (all four in Table 16). Stage numbers follow Fig. 1; "slots" is the share of dependency slots with a recovered producer; "at floor" counts families meeting the stage-2 support minimum ($\geq$ 5 groups on NESTFUL, $\geq$ 3 elsewhere); $n_{\max}$ is the largest family support (Eq. (7) requires 92); "p/a/w" is pass/abstain/wrong on the *family-level* replay split, not the 11/11 admission test split (Appendix F.3). AppWorld shows effect arm A.

Substrate	Traces	Slots (1)	Windows (2)	At floor (2)	Synth. (4)	Replay p/a/**w** (5)	$n_{\max}$ (6)	Adm.
NESTFUL	1,415	96.3%	1,207	32	12	24 / 12 / **0**	26	**0**
AppWorld	136	63.2%	136	33	1	34 / 0 / **0**	**136**	**1**

Each artifact was admitted at 92/92 zero-violation calibration groups ($U_{\hat{\eta}} = 0.0498 \leq 0.05$, conditional on i.i.d. groups), a support-sufficiency certificate (Section 3) that is compiler-wide for issue-type routing (one candidate at calibration) and per-candidate for PR-outcome and backlog routing (two; corrected $U = 0.0569$, so compiler-wide only at $\alpha \geq 0.057$; Corollary 2, Appendix C). The one certificate whose event includes the continuation comes from a post-cutoff PR-outcome cohort (pull requests created after the snapshot's last day; Appendix G): the re-derived artifact dispatched on 132 fresh calibration groups and the graded task contract missed 0, a one-sided bound of 0.0173 under the single rule pre-registered for that study and 0.035 under the primary grid. Those groups are 84 open and 48 merged pull requests and no closed-unmerged ones, so the certified population excludes that class; the 60 fresh test records (20 per class) pass under every condition, and hypothetical bounds using days and authors as independent units are 0.0228 and 0.0267; independence of those clusters is unverified.

On `gpt-6-luna` and Anthropic `claude-sonnet-5`, re-discovery from each model's own traces on the same cohorts re-derives the identical programs on two of the three families and refuses backlog routing (on Sonnet at 91 and 90 of 92 groups, the floor binding), with four compiled-only excerpt misses on Sonnet; transferring the retained artifacts to `gpt-6-luna` keeps the structure with one compiled-only miss in 90 (Appendix G).

5.2 Newer records within a snapshot: compile or retire

On the two-read cross-repository PR-outcome task (Section 4), GAC completes 120/120 exact contracts on paired held-out records across four of five repositories and retires `pytorch/pytorch` at admission, where no calibration group is accepted at any threshold (Appendix F). The compiled program ran on 80/120 records: every open-class record abstained on the induced `pr.state` hull (discovery pools held 0–3 open records) and ran the unchanged agent, so requests fall 44.4% (66.7% on dispatched records) and tokens 52.4%. A balanced three-repository rerun with all three classes in discovery, `pytorch/pytorch` included, dispatches on 180/180 and reaches 180/180 at 66.7% fewer requests and 78.4% fewer tokens (Table 15, Appendix F), consistent with a composition-driven refusal. The hand-written program is more efficient in the core protocol and tied in the rerun.

5.3 A public benchmark where GAC helps: BIRD

A live SQL agent on BIRD (Li et al., 2023a), a public text-to-SQL benchmark, shows where GAC helps (Appendix F.4): on the five dev databases with at least 146 questions, GAC retires all five when the agent picks its tables per question, and admits the schema-first opening on all five. On 150 sealed questions the compiled agent dispatched every time, answered 97 correctly against 94 (McNemar $p = 0.45$), and cut requests 44.9% and latency 29.4% against a warm rerun of the unchanged agent. Retrospective all-scheduled counts are 93 against 91 of 150 on a second model, and 402 against 403 of 630 on 21 of 26 training databases. Requests fall 49.6% and 45.8% on 150 and 629 completed pairs, respectively. A conservative paired 95% interval for the training difference is $[-3.27, +2.96]$ points under i.i.d. pairs; Table 22 reports database sensitivity. A second provider's agent never ran a fixed opening and was refused (Table 20).

5.4 Where GAC yields no measured benefit: NESTFUL and AppWorld

Thin evidence. NESTFUL, the largest trace corpus the compiler ingests here (1,415 traces), passes provenance, synthesis, and replay with *no* wrong execution, yet its best family reaches $n_{\max} = 26 <$

92, so every family retires at stage 6 (Table 2, Fig. 3 right); API-Bank and BFCL v4 retire the same way at 8 and 15 (Table 16).

Admissible, rarely dispatchable. AppWorld's gold solutions reach $n_{\max} = 136$, and GAC admits one argument-free two-call artifact at 92/92 zero-violation groups ($U_{\hat{\eta}} = 0.0498$, compiler-wide under one effect arm, per-candidate under the other; Appendix F.3). Yet across 8,190 released trajectories it is entry-eligible in 37.9%, almost always for full-code agents and almost never for ReAct and plan-and-execute, which open by reading API documentation (Table 17); admitting that read as a prologue recovers only 2 more of 4,680 (Appendix G).

5.5 Live recovery: request scope helps, completeness benefit is unresolved

A separate four-arm study tests a three-comment prefetch with equal recovery tools, removing request-scope admission or completeness checking one at a time; baseline receives no prefetch (Appendix I). All 2,304 scheduled episodes were attempted; four timed out. On 72 confirmation issue groups with constructed requests, full succeeds on 205/216 ordinary-response cases versus 186/216 without the scope guard: +8.80 points, descriptive issue-bootstrap 95% interval $[3.70, 13.89]$. Under capped responses, removing completeness checking yields 204/216 versus full's 205/216; the agent repairs 22/27 incomplete accepted prefetches. Full does not demonstrate an accuracy gain over baseline. In the separate 24-group replication, model cost falls 18.2% (ordinary) and 9.1% (capped), but tool calls rise and correct answers fall from 71 to 70 and 70 to 68 of 72. Hand-specified contracts, prior offline exposure, one model, and unadjusted intervals limit inference: this is mechanism evidence, not a new live-answer certificate or learned-gate benefit.

6 Related Work

Trace compilation and specialization. Beyond the trace-compilation precedent of the introduction, GAC takes meta-tracing's unit of specialization, the trace (Bolz et al., 2009). Partial evaluation is the older frame: a specialized program is correct only where its static assumptions hold (Futamura, 1971; Jones et al., 1993); here grounded arguments, declared effects (Lucassen & Gifford, 1988), and manifest pins are the guard, and each can veto.

Mining workflows from traces. Family mining is a narrow form of process mining (van der Aalst, 2016); synthesis is bounded programming by example and sketching (Gulwani, 2011; Solar-Lezama et al., 2006) with explicit provenance (Cheney et al., 2009); and likely invariants inferred from executions (Ernst et al., 2001) are the ancestor of the induced verifier, with the same caveat (Appendix G): an inferred hull is evidence, not proof.

Reuse, caching, and optimization of agents. Agent systems mine workflows, cache plans, compile orchestration, validate plans, or prune topology (Wang et al., 2025; Zhang et al., 2025; Abuzakuk et al., 2026; Wei et al., 2026; Winston et al., 2026; Li et al., 2026; Chen et al., 2026; Wang et al., 2026; Chundru, 2026); skill libraries grow reusable procedures from experience (Wang et al., 2024; Zheng et al., 2025); semantic caches reuse responses (Bang, 2023), and prompt caching and cache-aware compression reuse shared prefixes (Lumer et al., 2026; Song, 2026); DSPy, GEPA, TextGrad, and Trace optimize prompts and programs against task feedback (Khattab et al., 2024; Agrawal et al., 2026; Yuksekgonul et al., 2025; Cheng et al., 2024); RouteLLM changes models (Ong et al., 2025), LLMCompiler parallelizes calls (Kim et al., 2024), and Yue et al. (2026) survey the space. Our contribution combines trace reuse with a finite-sample admission bound and exact fallback on clean rejection; the reviewed systems motivate its components rather than establishing a universal priority claim. Counterfactual trace auditing (Zhou et al., 2026), which pairs each trace with a counterfactual run of the same task, is the closest analogue of GAC's continuation check.

Selective prediction and finite-sample risk control. Dispatch-or-fall-back is selective prediction with a reject option (Chow, 1970; Geifman & El-Yaniv, 2017). The admission bound is a fixed-grid instance of learn-then-test (Angelopoulos et al., 2025) using the exact binomial interval (Clopper & Pearson, 1934); the risk-controlling prediction-set framework gives related high-probability risk guarantees (Bates et al., 2021). Conformal prediction controls marginal coverage (Vovk et al., 2005);

conformal risk control bounds expected monotone loss (Angelopoulos et al., 2024), a different guarantee.

Tool-use safety and evaluation. ToolEmu and AgentDojo evaluate agents against hazardous or injected tool environments (Ruan et al., 2024; Debenedetti et al., 2024), the regime our perturbation studies probe (Appendix G). BFCL, ToolLLM, NESTFUL, API-Bank, AppWorld, ToolSandbox, and τ^2-bench score tool use broadly (Patil et al., 2025; Qin et al., 2024; Basu et al., 2025; Li et al., 2023b; Trivedi et al., 2024; Lu et al., 2025; Barres et al., 2025); GAC runs a live agent on BIRD (Li et al., 2023a) and uses four of these as compiler substrates, reporting what each refuses (Appendix F.3).

7 Discussion and Limitations

The supported claim is narrow: repeated read-only prefixes can become guarded programs that preserve held-out contracts on the tested GitHub families, show no detected accuracy loss on BIRD, and retire when evidence is inadequate. Across 810 held-out GitHub episodes run under a compiled artifact on the calibrated model in six studies, one produced a compiled-only contract failure: 1 of 270 on the excerpt-bearing contracts (one-sided bound 1.7%) and 0 of 540 on the outcome-only cross-repository contract (that subset covers 354 distinct records) (Appendix D); Appendix G pairs limitations with protocols and Appendix J maps claims to evidence.

What the certificates cover. The primary registered certificates are support-sufficiency statements about a replay-contract event without support here (Section 3): they certify neither graded selectivity of q — the cross-repository learned and support-only arms were indistinguishable, and an amended 184-group issue study retired (Appendix H) — nor the continuation, so the compiled-only misses lie outside them; the end-to-end certificate is per-candidate and covers one family. Every bound assumes i.i.d. groups: counting a creation day or an author as one unit retires every primary family under the registered grid (Table 24; single-rule sensitivity in Appendix C), and AppWorld's bound retires at scenario or supervisor level (Appendix F.3).

External validity, guards, and comparison. The richer evidence comes from one pinned snapshot, five frozen cross-repository snapshots, one post-cutoff cohort, one public benchmark (BIRD), and one calibrated model family (replications in Sections 5.1 and 5.3). In the six-study GitHub summary, every compiled-only miss is an excerpt departing from identical evidence, with no counterfactual attributing it to the model rather than the compiled context (Appendix G). Under drift we show no protection: two provider-free perturbation studies are null by construction (the simulated tools stay total; on the GitHub records the oracle is the derived call argument, fixed by the entry id), the live continuation-graded study is adverse (an emptied list lies inside the learned hull; both arms answered wrongly on 39 of 60 records under the original grader, 40 under its later correction), with no demonstrated protection in those studies (Appendix G). The separate constructed-request study supports request-scope admission, but not completeness checking or end-to-end discovery of the required contracts (Section 5.5). We do not price manual authoring or compare automated workflow-reuse systems head-to-head; the learned gate's incremental value remains unestablished. Writes, handoffs, hosted tools, undeclared effects, and removed-turn guardrails lie outside Proposition 1.

8 Conclusion

GAC treats recurrence as evidence, not permission: it saves resources while showing no detected accuracy loss in the primary GitHub and BIRD comparisons, certifies the end-to-end contract on one fresh cohort, refuses where evidence is thin, and shows request-scope protection on constructed live tasks. Completeness checking can duplicate the agent's own repair; earlier drift tests remain negative. The evidence supports bounded compilation and mechanism-specific safeguards, not universal quality preservation or learned-gate superiority.

AI Use Statement

The authors used generative AI tools to restructure and copy-edit the manuscript, suggest section organization, draft and revise paper text, critique methodological exposition and internal consistency,

convert repository-grounded results into LaTeX tables, and assist in designing and implementing the experiment and analysis scripts used to produce the reported results. The reported results distinguish live-provider execution, replay, provider-free audits, and simulation, and are linked to retained artifacts. The authors are responsible for the final content, experimental design, and reported results.

Ethics Statement

The main risk in removing model-mediated decisions is over-trusting a compiled prefix, and thereby accelerating unsafe automation or deleting a guardrail evaluation that happened to run at the removed boundary. The method mitigates this by compiling only read-only prefixes, treating unknown effects, approvals, handoffs, and writes as hard barriers, and defaulting to retirement or fallback when evidence is insufficient. Empirical inputs are public GitHub records and public benchmarks; separate simulation studies are labeled. Public records can include usernames and user-authored text; no private repository or private loan records were acquired. The artifact is not a license to bypass human review, legal or compliance controls, or safety guardrails; unsupported regimes are part of the result, not exceptions to hide.

Reproducibility Statement

Algorithm 2 gives the compiler and Alg. 1 the admission protocol, Algs. 3 and 4 give provenance construction and runtime dispatch, Appendix A proves Proposition 1, and Appendix C records the registered statistical and synthesis settings. The accompanying artifact contains source manifests, raw result files, deterministic table and figure generation, and scripts for the live GitHub studies, the cross-repository extension, the BIRD SQL-agent study with its extensions, the fresh-record and live-recovery safeguard studies (Appendix I), and all four compiler substrates: NESTFUL, API-Bank, executed BFCL, and executed AppWorld. Appendix F.3 states how the latter two obtain observed results, and Appendix C gives their exact provider-free entry points. Before upload, these materials will be supplied as an anonymous archive with hashes and manifests preserved. The artifact pins the provider model string, the `openai-agents` and `openai` SDK versions, the effective generation settings, cache policy, the 120 s timeout and no-retry policy, the per-record condition order, concurrency, the token fields used for cost, the rate-card date, the tool and effect-catalog digests, and every split and candidate manifest (Table 3); the second-model and second-provider replications pin their model strings, the Anthropic adapter's request mapping, and their protocols the same way. The added safeguard studies retain their configurations and execution schedules in separate prospective seals (Appendix I).

References

Sami Abuzakuk, Anne-Marie Kermarrec, Rishi Sharma, Rasmus Moorits Veski, and Martijn de Vos. Optimizing agentic workflows using meta-tools. *arXiv preprint arXiv:2601.22037*, 2026. URL `https://arxiv.org/abs/2601.22037`.

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026. URL `https://arxiv.org/abs/2507.19457`.

Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *International Conference on Learning Representations*, 2024. arXiv:2208.02814.

Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control. *The Annals of Applied Statistics*, 19(2):1641–1662, 2025. doi: 10.1214/24-AOAS1998. URL `https://arxiv.org/abs/2110.01052`.

Anthropic. Claude API pricing, 2026. URL `https://platform.claude.com/docs/en/about-claude/pricing`. Official documentation; accessed October 8, 2026.

Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. Dynamo: A transparent dynamic optimization system. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 1–12, 2000. doi: 10.1145/349299.349303.

Fu Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*, pp. 212–218, 2023. URL https://aclanthology.org/2023.nlposs-1.24/.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-Bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025. URL https://arxiv.org/abs/2506.07982.

Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate, Mayank Agarwal, Maxwell Crouse, Yara Rizk, Kelsey Bradford, Asim Munawar, Sadhana Kumaravel, Saurabh Goyal, Xin Wang, Luis A. Lastras, and Pavan Kapanipathi. NESTFUL: A benchmark for evaluating LLMs on nested sequences of API calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 33538–33547, 2025. doi: 10.18653/v1/2025.emnlp-main.1702. URL https://aclanthology.org/2025.emnlp-main.1702/.

Stephen Bates, Anastasios N. Angelopoulos, Lihua Lei, Jitendra Malik, and Michael I. Jordan. Distribution-free, risk-controlling prediction sets. *Journal of the ACM*, 68(6), 2021. doi: 10.1145/3478535. URL https://arxiv.org/abs/2101.02703. Article 43.

Carl Friedrich Bolz, Antonio Cuni, Maciej Fijałkowski, and Armin Rigo. Tracing the meta-level: PyPy's tracing JIT compiler. In *Proceedings of the 4th Workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems (ICOOOLPS)*, pp. 18–25, 2009. doi: 10.1145/1565824.1565827.

Yulang Chen, Haoxuan Peng, Jinyan Liu, Zichen Wen, Dongrui Liu, and Linfeng Zhang. AgentSlimming: Towards efficient and cost-aware multi-agent systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pp. 30064–30086, 2026. doi: 10.18653/v1/2026.acl-long.1387. URL https://aclanthology.org/2026.acl-long.1387/.

James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009. doi: 10.1561/1900000006.

Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the next AutoDiff: Generative optimization with rich feedback, execution traces, and LLMs. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2406.16218.

C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. doi: 10.1109/TIT.1970.1054406.

Jagadeesh Chundru. Agentic compilation: Mitigating the LLM rerun crisis for minimized-inference-cost web automation. *arXiv preprint arXiv:2604.09718*, 2026. URL https://arxiv.org/abs/2604.09718.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993. doi: 10.1201/9780429246593.

Michael D. Ernst, Jake Cockrell, William G. Griswold, and David Notkin. Dynamically discovering likely program invariants to support program evolution. *IEEE Transactions on Software Engineering*, 27(2):99–123, 2001. doi: 10.1109/32.908957.

Yoshihiko Futamura. Partial evaluation of computation process—an approach to a compiler-compiler. *Systems, Computers, Controls*, 2(5):45–50, 1971. URL https://fi.ftmr.info/PE-Museum/PE-Original-English1971.pdf.

Andreas Gal, Brendan Eich, Mike Shaver, David Anderson, David Mandelin, Mohammad R. Haghighat, Blake Kaplan, Graydon Hoare, Boris Zbarsky, Jason Orendorff, Jesse Ruderman, Edwin W. Smith, Rick Reitmaier, Michael Bebenita, Mason Chang, and Michael Franz. Trace-based just-in-time type specialization for dynamic languages. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 465–478, 2009. doi: 10.1145/1542476.1542528.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html.

Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pp. 317–330, 2011. doi: 10.1145/1926385.1926423.

Headroom Labs. Headroom: Compressing tool outputs, logs, and retrieved context before they reach the LLM. Software repository, https://github.com/headroomlabs-ai/headroom, 2026. Apache-2.0; self-reported 60–95% token reduction on JSON payloads and 73% on a GitHub triage workload. Accessed 7 August 2026.

helmo. GitHub issues dataset card. Hugging Face Datasets, 2025. URL https://huggingface.co/datasets/helmo/github-issues. Revision e344be7b84d199661a9956036991e1fc25715a47.

Urs Hölzle, Craig Chambers, and David Ungar. Debugging optimized code with dynamic deoptimization. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 32–43, 1992. doi: 10.1145/143095.143114. URL https://bibliography.selflanguage.org/dynamic-deoptimization.html.

Neil D. Jones, Carsten K. Gomard, and Peter Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993. URL https://studwww.itu.dk/people/sestoft/pebook/.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/f1cf02ce09757f57c3b93c0db83181e0-Paper-Conference.pdf.

Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 24370–24391, 2024. URL https://proceedings.mlr.press/v235/kim24y.html.

LangChain. Build a SQL agent, 2026. URL https://docs.langchain.com/oss/python/langchain/sql-agent. Official documentation; accessed October 8, 2026.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM already serve as a database interface? A BIg Bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023a. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/83fc8fab1710363050bbd1d4b8cc0021-Abstract-Datasets_and_Benchmarks.html.

Junyan Li, Zhang-Wei Hong, Maohao Shen, Yang Zhang, and Chuang Gan. FlowCompile: An optimizing compiler for structured LLM workflows. *arXiv preprint arXiv:2605.13647*, 2026. URL https://arxiv.org/abs/2605.13647.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 3102–3116, 2023b. doi: 10.18653/v1/2023.emnlp-main.187. URL https://aclanthology.org/2023.emnlp-main.187/.

Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025. doi: 10.18653/v1/2025.findings-naacl.65. URL https://aclanthology.org/2025.findings-naacl.65/.

John M. Lucassen and David K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 47–57, 1988. doi: 10.1145/73560.73564.

Elias Lumer, Faheem Nizar, Akshaya Jangiti, Kevin Frank, Anmol Gulati, Mandar Phadate, and Vamse Kumar Subbiah. Don't break the cache: An evaluation of prompt caching for long-horizon agentic tasks. *arXiv preprint arXiv:2601.06007*, 2026. URL https://arxiv.org/abs/2601.06007.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12:153–157, 1947. doi: 10.1007/BF02295996.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs from preference data. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html.

OpenAI. Agents SDK guide. OpenAI Developer Documentation, 2026a. URL https://developers.openai.com/api/docs/guides/agents. Accessed 2026-08-03.

OpenAI. API pricing. OpenAI Developer Documentation, 2026b. URL https://developers.openai.com/api/docs/pricing. Accessed 2026-08-03.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2307.16789.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*, 2024. arXiv:2309.15817.

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 404–415, 2006. doi: 10.1145/1168857.1168907.

Yan Song. Cache-aware prompt compression: A two-tier cost model for LLM API caching. *arXiv preprint arXiv:2607.15516*, 2026. URL https://arxiv.org/abs/2607.15516.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076, 2024. doi: 10.18653/v1/2024.acl-long.850. URL https://aclanthology.org/2024.acl-long.850/.

Wil M. P. van der Aalst. *Process Mining: Data Science in Action*. Springer, second edition, 2016. doi: 10.1007/978-3-662-49851-4.

Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005. URL https://link.springer.com/book/10.1007/b106715.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2305.16291.

Jincheng Wang, Min Zheng, and Tao Wei. COVENANT: Natural-language workflow compilation for aligned agent execution. *arXiv preprint arXiv:2607.25400*, 2026. URL https://arxiv.org/abs/2607.25400.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 63897–63911, 2025. URL https://proceedings.mlr.press/v267/wang25bx.html.

Lei Wei, Qi Liu, Ruiyang Huang, Xiao Peng, Sicong Xie, Lanbo Lin, Chenhao Jiang, Yuanwu Xu, Tianyuan Yang, Jiayao Liu, Li Cai, Zhaolu Kang, and Bin Wang. EvoC2F: Compiling tool orchestration for efficient and evolvable LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://proceedings.mlr.press/v306/wei26q.html.

Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.

Caleb Winston, Ron Yifeng Wang, Azalia Mirhoseini, and Christos Kozyrakis. Agent JIT compilation for latency-optimizing web agent planning and scheduling. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://proceedings.mlr.press/v306/winston26b.html.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Ling Yue, Kushal Raj Bhandari, Ching-Yun Ko, Dhaval Patel, Shuxin Lin, Nianjun Zhou, Jianxi Gao, Pin-Yu Chen, and Shaowu Pan. From static templates to dynamic runtime graphs: A survey of workflow optimization for LLM agents. *arXiv preprint arXiv:2603.22386*, 2026. URL https://arxiv.org/abs/2603.22386.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative AI by backpropagating language model feedback. *Nature*, 639:609–616, 2025. doi: 10.1038/s41586-025-08661-4. URL https://www.nature.com/articles/s41586-025-08661-4.

Qizheng Zhang, Michael Wornow, and Kunle Olukotun. Agentic plan caching: Test-time memory for fast and cost-efficient LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/9549f7d06700f0966d5f938f1d11022a-Abstract-Conference.html.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zheng Zhuoyang Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.

Xiaolin Zhou, Jinbo Liu, Li Li, Ryan A. Rossi, and Xiyang Hu. Counterfactual trace auditing of LLM agent skills. *arXiv preprint arXiv:2605.11946*, 2026. URL https://arxiv.org/abs/2605.11946.

A Proof of Proposition 1

Condition on the train/development data, fix $\eta \in \Lambda$, and write $\gamma = \delta/|\Lambda|$. Let D_i^η and W_i^η be the group admission and violation indicators defined in Section 2. Because the entire candidate and grid were frozen before calibration and groups are i.i.d., the pairs (D_i^η, W_i^η) are i.i.d. Conditional on $n_\eta > 0$ admitted groups, their violation count therefore satisfies $k_\eta \mid n_\eta \sim \text{Binomial}(n_\eta, r_\eta)$, where $r_\eta = \Pr(W^\eta = 1 \mid D^\eta = 1)$. Inverting the one-sided exact binomial test (Clopper & Pearson, 1934) gives, for $0 \le k_\eta < n_\eta$, the upper bound $U_\eta = \text{Beta}^{-1}(1-\gamma;\, k_\eta+1,\, n_\eta-k_\eta)$ of Eq. (7), with $U_\eta = 1$ when $k_\eta = n_\eta$; either way $\Pr\{r_\eta > U_\eta \mid n_\eta\} \le \gamma$. When $n_\eta = 0$ the event is impossible under the convention that empty-admission thresholds receive the vacuous bound $U_\eta = 1$. Averaging over the random value of n_η therefore yields $\Pr\{r_\eta > U_\eta\} \le \gamma$ for each fixed threshold. Applying the union bound over the finite grid gives $\Pr\{\exists \eta \in \Lambda : r_\eta > U_\eta\} \le |\Lambda|\gamma = \delta$, i.e. simultaneous coverage with probability at least $1 - \delta$. On that event, every threshold whose upper bound is at most α satisfies the group-level target in Eq. (4), including the threshold returned by Alg. 1. ■

Two scope conditions are worth restating. First, the statement is *per fixed candidate*: Alg. 2 may calibrate several families and retain survivors, and that outer search is not multiplicity-corrected — except when the search is frozen, per the corollary below. Second, i.i.d. source groups is an assumption about the calibration cohort, not a property the compiler verifies; the grouped split in Alg. 2 prevents within-group leakage but does not establish independence across groups or under temporal drift.

Corollary 2 (Compiler-wide guarantee under frozen selection)**.** *If Alg. 2 runs with candidate freezing enabled — ranking on train data, synthesizing and challenging on train and development windows only, and halting the search the instant any candidate reaches calibration, whatever its outcome — Proposition 1's guarantee holds for the run's actual reported output with no adjustment to $\gamma = \delta/|\Lambda|$: no candidate multiplicity remains to correct for.*

Proof. Ranking, synthesis, the challenge test, and the frozen score model q are all functions of train and development windows alone, fixed before a single calibration window is inspected. The search reaches calibration for the top-ranked surviving candidate exactly once and halts immediately after that candidate's outcome, so no second candidate's program, guard, verifier, or score model is ever compared against calibration data. Proposition 1's precondition — one candidate fixed before observing calibration — is met by the run's entire search, not by a hypothetical single-candidate sub-procedure, so its conclusion transfers unchanged. ■ □

This is a corollary, not a new inequality: it is a restatement of Proposition 1 for a search that never lets a second candidate see calibration data. Conditional on train and development data, the number m of candidates that reach calibration is fixed before any calibration label is read. The cross-repository extension runs with freezing enabled and every sealed repository report shows one candidate reaching calibration, so Corollary 2 applies to those admissions directly. Issue-type routing did not freeze, but its second candidate retired at synthesis (an ungroundable slot) and never saw calibration, so $m = 1$ and the same argument applies. PR-outcome and backlog-attention routing each calibrated $m = 2$ candidates on the same 92 groups and kept the higher-removal survivor: their certificates are per-candidate at $\alpha = 0.05$, and under $\gamma = \delta/(m|\Lambda|)$ the retained zero-violation tables give $U = 0.0569 > \alpha$, a compiler-wide certificate only at $\alpha \ge 0.057$ or after 106 groups (Table 4). The pre-registered repair, a fresh class-balanced 106-group calibration per family (`paper/supplementary/multiplicity-repair-protocol.md`), failed its own go/no-go check: after excluding every record used in a retained cohort, the pinned snapshot holds 33 unused `open` pull requests and 8 unused `owned` backlog issues against the 36 per class the design requires, so it was not run and the per-candidate wording stays.

Multiplicity accounting. For one fixed candidate and one fixed threshold η, $\Pr(r_\eta > U_\eta) \le \gamma = \delta/|\Lambda|$. For one fixed candidate over the whole grid, the union bound gives $\Pr(\exists \eta \in \Lambda : r_\eta > U_\eta) \le |\Lambda|\gamma = \delta$. For a set of m candidates fixed before calibration and each evaluated over Λ, choosing $\gamma = \delta/(m|\Lambda|)$ gives $\Pr(\exists\, a, \eta : r_{a,\eta} > U_{a,\eta}) \le \delta$; at $\alpha = 0.05$, $\delta = 0.1$, $|\Lambda| = 11$ the zero-violation requirement is 92 groups for $m = 1$ and 106 for $m = 2$ ($U = 0.0498$ versus 0.0569 at $n = 92$). `paper/scripts/frozen_candidate_coverage_simulation.py` checks the *single-threshold* quantity exactly, without Monte Carlo: for one candidate at one threshold and $n = 92$, the

Algorithm 2 COMPILE — the compile-or-retire cascade. Every candidate is either registered or assigned an attributed RETIRE, so refusal accounting remains explicit. Stages marked (CALLER) are separate library entry points run in this order rather than steps inside one function.

Input: episodes $\mathcal{E}$; effect catalog C; entry schema Θ; transform library $\mathcal{L}$; ambiguity cap κ; mining window $w_{\max}$; budgets α, δ; grid Λ; feasibility target Δ
Output: artifact set $\mathcal{A}$, or RETIRE with attributed reasons

$\mathcal{A} \leftarrow \emptyset$
$\mathcal{E} \leftarrow \textsc{Qualify}(\mathcal{E}, C)$; $\{\mathcal{E}_M\} \leftarrow \textsc{PartitionBy}(\mathcal{E}, \text{manifest, principal, isolation})$ ▷ drop partial runs, unpinned manifests
for all partitions $\mathcal{E}_M$ **do**
  $(\mathcal{T}, \mathcal{D}, \mathcal{K}, \mathcal{S}) \leftarrow \textsc{GroupedSplit}(\mathcal{E}_M)$ (CALLER) ▷ train / dev / calibration / sealed test, split by *group*
  $(G, S) \leftarrow \textsc{BuildPatg}(\mathcal{T}, \Theta, C, \mathcal{L}, \kappa)$ ▷ Alg. 3: annotate every slot
  $\mathcal{F} \leftarrow \textsc{Canonicalize}(\textsc{MineRegions}(G, S, C, w_{\max}))$, keeping only cross-group support ▷ rejects any window with an UNGROUNDED or AMBIGUOUS slot
  if $\textsc{Ceiling}(\mathcal{T}, \mathcal{F}) < \Delta$ **then** (CALLER)
    record RETIRE **(infeasible ceiling, Eq. (6))**; **continue** ▷ try the next compatible partition
  end if
  for all families $F \in \mathcal{F}$ ranked by Eq. (5) **do**
    $P \leftarrow \textsc{Synthesize}(F, \mathcal{L})$ at depth ≤ 2; **if** $P = \perp$ **then record** RETIRE **(synthesis); continue**
    $(H, V) \leftarrow \textsc{InduceContract}(F)$; **if** $\neg\textsc{ReplayAndChallenge}(P, V, \mathcal{D})$ **then record** RETIRE **(counterexample); continue**
    $q \leftarrow \textsc{FitScore}(\mathcal{D})$; **freeze** q ▷ dev groups only; never calibration
    $\eta \leftarrow \textsc{Calibrate}(P, H, V, q, M, \mathcal{K}, \Lambda, \alpha, \delta)$; **if** $\eta = \perp$ **then record** RETIRE **(risk); continue** ▷ Alg. 1
    $\mathcal{A} \leftarrow \mathcal{A} \cup \{\textsc{Package}(P, H, V, q, \eta, M)\}$ with evidence and lifecycle
  end for
end for
return $\textsc{SelectUndominated}(\mathcal{A})$ if nonempty, else RETIRE

population rate that maximizes miscoverage is $r^* \approx 0.637$, where miscoverage is $\approx 0.00909 \approx \gamma$. Pooling m such candidates at the uncorrected per-candidate budget raises that single-threshold union to 1.8% at $m = 2$ and 13.6% at $m = 16$, while the corrected budget keeps it below 0.81% for every $m \ge 2$ tested ($m = 1$ sits at the 0.91% per-threshold budget by construction). These are illustrative single-threshold figures; the grid-wide and candidate-wide certificates are the δ statements above, not these numbers. A companion seeded Monte Carlo (seed 20260822, 200,000 replicates per cell) draws the 92 groups in correlated blocks instead of i.i.d.: realized miscoverage matches the exact value at zero correlation and reaches 56% against the 0.9% nominal single-threshold budget at the most clustered configuration tested (block size 46, one quarter of blocks shifted by 0.3), which is why the live studies' `min_days=min_principals=1` configuration is this guarantee's weakest link.

B Additional Algorithms

Algorithm 3 constructs the typed provenance graph that decides Eq. (1). PATG annotates; MINEREGIONS blocks. The two blocking marks are asymmetric on purpose: a slot with no witness is a genuine model decision and a slot with more than κ witnesses is ambiguous, so the analysis never resolves an argument by picking the cheapest available explanation. PATG keeps up to κ single-source witnesses per grounded slot and SYNTHESIZE commits to the one source stable across every supporting group.

Algorithm 4 is the runtime counterpart of Eq. (3). Cheap deterministic checks reject first and the calibrated gate runs only on what survives them, so the common case costs a registry lookup and a guard evaluation. Once an artifact resolves, the position check runs before any guard: any tool already observed in the episode, whether or not the compiled program uses it, returns the unchanged agent (Eq. (2)). Exactly one exit path compacts; every clean rejection returns the unmodified baseline agent, and any failure whose reversibility cannot be attested raises an incident rather than feigning a rollback. "Baseline" is byte-exact only where a staging owner holds the commit boundary: the outer runner can restore byte-identical model input, whereas a model-boundary adapter cannot un-emit a response the host has already committed to conversation history.

C Configuration

Live-provider configuration. Every primary live study calls one provider and one model: OpenAI `gpt-5.6-luna` through the `openai-agents` SDK's Responses interface. Model settings are fixed across all conditions at reasoning effort `low` and verbosity `low`, with parallel tool calls disabled so provider-visible interface counts are comparable between the baseline and a compiled prefix, and server-side response storage disabled. Sampling parameters are not exposed for this model; the

Algorithm 3 BUILDPATG — typed argument provenance. PATG annotates every argument slot and blocks nothing itself; MINEREGIONS (Alg. 2) rejects any window containing an UNGROUNDED or AMBIGUOUS slot, so a slot with no witness (unresolved in the bounded library) and a slot with too many witnesses both retire the region rather than defaulting to the cheapest explanation.

Input: episodes $\mathcal{T}$; entry schema Θ; effect catalog C; transform library $\mathcal{L}$; ambiguity cap κ
Output: provenance graph G (witness and order edges) and slot-status map S

$G \leftarrow \emptyset$; $S \leftarrow \emptyset$
for all episodes $E = (z, M, e_{1:T}, y) \in \mathcal{T}$ **do**
  $\Sigma \leftarrow \{(\text{“}z\text{”}, \pi, v) : (\pi, v) \in \text{FLATTEN}(z),\ \pi \in \Theta\}$ ▷ only allowlisted entry paths are eligible
  for all call–result pairs (c_j, o_j) in $e_{1:T}$, in temporal order **do**
    for all slot names $s \in \text{keys}(\text{args}(c_j))$ **do**
      $v_{j,s} \leftarrow \text{args}(c_j)[s]$ ▷ the slot's value, distinct from its name
      if (c_j, s) is declared literal-only in Θ **then**
        $S[j, s] \leftarrow$ LITERAL; **continue** ▷ preserve the declared constant; infer no provenance
      end if
      $\mathcal{H} \leftarrow \text{BESTPERSOURCE}\{(\sigma, g) \in \Sigma \times \mathcal{L} : g(\text{value}(\sigma)) = v_{j,s},\ \text{depth}(g) \le 2\}$
      if $\mathcal{H} = \emptyset$ **then**
        $S[j, s] \leftarrow$ UNGROUNDED ▷ no witness: a model decision
      else if $|\mathcal{H}| > \kappa$ **then**
        $S[j, s] \leftarrow$ AMBIGUOUS
      else
        $S[j, s] \leftarrow$ GROUNDED; $G \leftarrow G \cup \{\sigma \xrightarrow{g} (j, s) : (\sigma, g) \in \mathcal{H}\}$ ▷ $1..\kappa$ single-source witnesses; synthesis commits to one
      end if
    end for
    $\Sigma \leftarrow \Sigma \cup \{(\text{var}(c_j), \pi, v) : (\pi, v) \in \text{FLATTEN}(o_j),\ \text{ELIGIBLE}(\pi, v)\}$ ▷ o_j witnesses only *later* slots
  end for
  for all event pairs (e_s, e_t) with $s < t$ sharing a resource where at least one writes it **do**
    $G \leftarrow G \cup \{e_s \prec e_t\}$ ▷ order edge in observed temporal order
  end for
end for
return (G, S)

SDK sends none, and the effective request manifest is Table 3. The replications in Appendix G run the same harness on `gpt-6-luna` (same settings) and on Anthropic `claude-sonnet-5` through an adapter over the official SDK (adaptive thinking at effort `low`, parallel tool use disabled, structured answers through the same schemas). Reported dollars are an *estimate*, computed by applying the standard short-context list rates below (OpenAI, 2026b; Anthropic, 2026) — retrieved 2026-08-02 for `gpt-5.6-luna` and 2026-09-24 for the other two — to the token counts retained per record; they are not an account invoice and carry no negotiated or cached-tier discount beyond the cached-input line.

Rate (USD per million tokens)	`gpt-5.6-luna`	`gpt-6-luna`	`claude-sonnet-5`
Input	0.20	0.10	2.00
Cached input	0.02	0.01	0.20
Cache write	0.25	0.125	2.50
Output	1.20	0.50	10.00

Because every condition within a family shares this model, these settings, the same records, and the same grader, the conditions differ in execution structure alone.

Admission budgets. The primary registered-grid admission runs use risk budget $\alpha = 0.05$, confidence budget $\delta = 0.1$, and a fixed grid of $|\Lambda| = 11$ thresholds, which fixes the zero-violation sample requirement at $n_\eta \ge \ln(\delta/|\Lambda|)/\ln(1-\alpha) = 91.6$, i.e. 92 admitted groups; at exactly $n_\eta = 92$ and $k_\eta = 0$ the bound is $U_\eta = 1 - (0.1/11)^{1/92} = 0.0498 \le \alpha$. Tightening the confidence budget to $\delta = 0.05$ would raise the requirement to 106 groups, beyond these 92-group pools. The later 132-group end-to-end study uses a prospectively frozen single rule; exploratory runs with other budgets are labeled separately. A Bonferroni repair over $m = 2$ candidate families uses $\gamma = \delta/(m|\Lambda|)$ and lands on the same 106-group requirement, which is why PR-outcome and backlog routing carry per-

Algorithm 4 DISPATCH — boundary-time admission. Cheap deterministic checks reject first and the calibrated gate runs only on what survives them. Exactly one exit path compacts; clean rejections return the unmodified agent, while an unattested failure or a refused commit raises an incident.

Input: entry state z; runtime context $X = (M', O, \text{SNAP}, \text{STAGE})$ — runtime manifest, tools already observed in this episode, optional snapshot provider, staging owner; registry $\mathcal{R}$; effect catalog C; mode $m \in \{\text{off}, \text{shadow}, \text{live}\}$
Output: observations for the region, FALLBACK to baseline agent B, or INCIDENT

if $m = \text{off}$ **then**
  return FALLBACK ▷ conformance: model-visible input must be byte-identical
end if
$A \leftarrow \mathcal{R}.\text{RESOLVE}(X.M'.\text{compatibility}, X.M'.\text{partition})$
if $A = \bot \lor A.\text{lifecycle} \neq \text{ACTIVE} \lor (\text{SIGNATUREREQUIRED} \land \neg \text{VERIFYSIGNATURE}(A))$ **then**
  return FALLBACK
end if
if $X.O \neq \emptyset$ **then**
  return FALLBACK ▷ Eq. (2): any earlier tool call, by any tool, leaves position 0
end if
if $H_A(z, X.M') \neq 1$ **then**
  return FALLBACK ▷ manifest pins, isolation keys, typed hulls, effect set
end if
if $m = \text{live} \land \exists f \in \text{tools}(A.P) : C(f).\textit{quota_attested} \land X.\text{SNAP} = \bot$ **then**
  return FALLBACK ▷ a quota-bearing read cannot be discarded cleanly without a snapshot
end if
if $q_A(z) > \eta_A$ **then**
  return FALLBACK ▷ calibrated abstention on an uncertain input
end if
if $m = \text{shadow}$ **then**
  log would-dispatch; **return** FALLBACK ▷ observe coverage without changing behavior
end if
$S \leftarrow X.\text{STAGE}.\text{BEGIN}(X.\text{SNAP})$; $r \leftarrow \text{INTERPRET}(A.P, z, \text{FACADE}(C))$ ▷ the facade re-checks effects at execution time
if r raised a pre-commit error **then**
  return FALLBACK if S.ABORTANDATTEST(), else INCIDENT
end if
if $V_A(r) \neq 1$ **then**
  return FALLBACK if S.ABORTANDATTEST(), else INCIDENT ▷ live-outs, effect multiset, and call count are checked before use
end if
return r.observations if S.COMMIT(r.effects) succeeds, else INCIDENT ▷ commit refuses any effect outside the read classes; a refused commit is never a fallback

candidate certificates while issue-type routing and the frozen cross-repository families are compiler-wide (Table 4). The grid is $\Lambda = \{0.02, 0.05, 0.08, 0.11, 0.14, 0.17, 0.20, 0.25, 0.30, 0.40, 0.50\}$ and the ambiguity cap is $\kappa = 3$. Synthesis uses the 23-operator closed library of Table 6 at depth ≤ 2; region mining uses $w_{\max}$ tool calls per window and requires support across independent scenario groups. Every bound in this paper is conditional on i.i.d. calibration groups.

Calibration labels as computed. For the replay-calibrated artifacts, Alg. 1 receives labels produced by clean replay of the frozen program over each calibration group's recorded tool results (`grc/compile.py`, `_calibration_samples`). A group whose entry state fails the frozen hard guard is ineligible and never enters n_η. An eligible group enters n_η at every η its score permits, whatever happens next. If the interpreter fails or the verifier rejects, the group is *unproductive*: a clean abstention that stays in n_η with $L = 0$ and would run the unchanged agent at runtime. Only when the program completes and the verifier accepts is each live-out compared with the recorded result under the tool's declared equivalence; a missing or non-equivalent output is the violation counted in k_η. The downstream continuation is never executed in this path, so an answer-level miss after a clean region cannot enter k_η; it is measured on held-out records instead (Section 5.1, Appendix D). Across the retained pools every admitted artifact has $k_\eta = 0$; verifier abstentions do occur in some pools and are reported as unproductive, not as violations. The incident branch of L (a dirty failure after a commit) cannot arise in replay calibration of a read-only region and is enforced at runtime by staging (Alg. 4).

Table 3: Effective live-provider request manifest. Every row names its source: a retained `run` block, or a code constant with its file and line. Cache and cost fields are the ones the price formula consumes.

Key	Value
Provider / model	OpenAI `gpt-5.6-luna` via the `openai-agents` Responses interface
SDK versions	`openai-agents 0.19.2`, `openai 2.52.0`, Python 3.14.4, macOS-26.5.2 arm64
Model settings	reasoning effort `low`, verbosity `low`, `parallel_tool_calls=False`, `store=False`
Sampling	temperature, top-p, and seed are not exposed for this model; the SDK sends none
Turn and time limits	`max_turns` 8 (families) / 6 (cross-repository); 120 s per episode; no retries; failures kept in `failures[]`
Concurrency	discovery 6 (PR, backlog) / 8 (issue-type, cross-repository); held-out arms one at a time per record (PR, backlog, cross-repository); blocked Latin design, 8-way (issue-type)
Condition order	per-record permutation cycle over all six arm orders, five records per order
Cache accounting	fields `input_tokens`, `cached_tokens`, `cache_write_tokens`, `output_tokens`; no shared server state; cached tokens 0 in PR/backlog/cross-repository, non-zero in issue-type
Pricing	USD per 1M tokens: input 0.20, cached 0.02, cache write 0.25, output 1.20 (list rates, 2026-08-02); cost = (input − cached − write)·0.20 + cached·0.02 + write·0.25 + output·1.20
Data pins	`helmo/github-issues@e344be7b` parquet sha256 `09453eef…` (tracked); four cross-repository parquets pinned by sha256 in `source_manifest.json`
Tool and effect pins	per-family `effect_catalog_version` digest, `tools_hash`, `prompt_hash`, `policy_hash`, `entry_contract_version`
Run timestamps	one per study: 2026-08-04, 2026-08-05, 2026-08-05 (families); 2026-08-07 (core); 2026-08-22 (gate frontier); per-record timestamps not retained

Table 4: Certificate level of every primary artifact. m is the number of candidates that reached calibration, fixed conditional on train and development data; U is the zero-violation upper bound at $n = 92$ under the per-candidate budget $\gamma = \delta/|\Lambda|$ and under the candidate-wide budget $\gamma = \delta/(m|\Lambda|)$. Rows are recomputed from the retained per-threshold tables (`compiler.artifact.gate.notes`) of the admitted artifacts. A provider-free recompilation from the sealed discovery checkpoints reproduces both admitted artifacts (identical ids, split digests, and bounds) and recovers the dominated two-read candidates' tables, which are also 92/0 at every $\eta \geq 0.14$ with $U = 0.0498$ (`paper/results/iclr_revision/recompiled_candidates_*.json`).

Family	m	n_η/k_η	$U_{\delta/\|\Lambda\|}$ / $U_{\delta/(m\|\Lambda\|)}$	Certificate at $\alpha = 0.05$	Groups needed
Issue-type routing	1	92/0	0.0498 / 0.0498	compiler-wide	92
PR-outcome audit	2	92/0	0.0498 / 0.0569	per-candidate; compiler-wide at $\alpha \geq 0.057$	106
Backlog-attention routing	2	92/0	0.0498 / 0.0569	per-candidate; compiler-wide at $\alpha \geq 0.057$	106
Cross-repository (7 admitted)	1	92/0	0.0498 / 0.0498	compiler-wide (frozen search)	92

Retrospective single-rule sensitivity; registered certificates unchanged. Because the admitted artifacts in this sensitivity analysis have coverage 1, the operative rule on these cohorts is "dispatch every hard-guard-eligible group", and the 11-point grid buys nothing but its Bonferroni factor. Table 5 recomputes every retained bound as if a single acceptance rule had been pre-registered instead, so the confidence budget is split only across the m candidates that reached calibration ($\gamma = \delta/m$). Under that rule the zero-violation floor is 45 groups for $m = 1$ and 59 for $m = 2$ (77 and 93 with one violation), the $n = 92$ bounds are 0.0247 and 0.0320, and every primary and cross-repository artifact would be compiler-wide at $\alpha = 0.05$; the cluster-level bounds at distinct days admit everywhere and at distinct authors admit everywhere except `streamlit/streamlit` (25 authors, 0.0880). This rule was *not* registered for these runs, so the certificates in Table 4 stand as printed; the table is a sensitivity analysis, not a repair, and the author- and day-level columns inherit every caveat of Table 24: distinct counts do not establish independent clusters, and a cluster bound targets a different population from the record-level one. A single frozen rule is adopted only prospectively, for studies whose candidates,

Table 5: Retrospective single-rule sensitivity. Each row recomputes the retained all-eligible calibration table (n/k from the widest row of `compiler.artifact.gate.notes`) under one pre-registered acceptance rule with $\gamma = \delta/m$; "registered" is the grid certificate as printed. Day and author columns apply the same rule to the distinct-unit counts of Table 24 (count in parentheses). Hypothetical bounds for the already selected artifacts; not new certificates.

Cohort	m	n/k	U registered	U single, δ/m	single, days	single, authors
Issue-type routing	1	92/0	0.0498	0.0247	0.0253 (90)	0.0277 (82)
PR-outcome audit	2	92/0	0.0498	0.0320	0.0342 (86)	0.0487 (60)
Backlog-attention routing	2	92/0	0.0498	0.0320	0.0338 (87)	0.0402 (73)
Core: `huggingface/datasets`	1	92/0	0.0498	0.0247	0.0284 (80)	0.0488 (46)
Core: `pandas-dev/pandas`	1	92/0	0.0498	0.0247	0.0247 (92)	0.0425 (53)
Core: `psf/requests`	1	92/0	0.0498	0.0247	0.0253 (90)	0.0287 (79)
Core: `streamlit/streamlit`	1	92/0	0.0498	0.0247	0.0261 (87)	0.0880 (25)

acceptance policy, grouping and endpoint are fixed before fresh calibration; the post-cutoff study of Appendix G did so after this analysis and reports its bound under both rules. The symmetric pessimistic alternatives are not in the table: $\delta = 0.05$ raises the floor to 106 groups, and month-level units retire every cohort (`paper/results/iclr_revision/effective_units.json`).

Runtime capability vocabulary. `quota_attested` is a boolean field of a tool's signed effect declaration, not a capability flag: it marks a read whose invocation advances a counter (a rate quota or an audit log) that the staging snapshot must attest over before an attempt can be discarded as clean, so in live mode such a tool is not dispatched without a snapshot provider (Alg. 4). Position 0 of Eq. (2) is identified at runtime with "no tool observation has been committed in this episode": any earlier tool call, whether or not the compiled program uses that tool, returns the unchanged agent, the runtime form of the compile-time `prefix_only` constraint. In the shipped facade every effect outside {PURE, READ-LOCAL, READ-EXTERNAL} is refused at commit and raises an incident; no post-commit path is reachable in the reported runs, and Alg. 4's commit line is that contract.

Table 6: The closed transform library $\mathcal{L}$ (version `T-v1`). These 23 operators are the *entire* expressive power of the binder: a slot with no chain of length ≤ 2 over a single source is `ungroundable` and retires its region, so this table fixes what Eq. (1) can witness. Parameters shown in parentheses are drawn from the observed trace, not free-searched. Every chain is typed, and a runtime type mismatch prunes it during the value-directed search rather than at apply time.

Class	n	Operators
Identity / coercion	5	`id`, `str`, `int`, `float`, `bool`
String	6	`lower`, `upper`, `strip`, `split(sep)[i]`, `join(sep)`, `fmt(template)`
Numeric	5	`add(c)`, `mul(c)`, `round(k)`, `sum`, `len`
Collection	6	`project(path)`, `filter(path op const)`, `first`, `last`, `sort(path)`, `topk(path,n)`
Temporal	1	`date_fmt(pattern)`
Total	23	

The library is closed and versioned deliberately. Every operator added widens Alg. 3's coverage and its spurious-match surface together, so growth is a two-sided trade rather than a free improvement; artifacts pin the library version and a runtime version mismatch is a dispatch barrier. The search is value-directed rather than an enumeration of $|\mathcal{L}|^2$ expressions: it runs breadth-first from the concrete source value and keeps chains whose denotation equals the target, which prunes by runtime type for free and collapses observationally equivalent chains.

The nonconformity score q. q is an ℓ_2-regularized logistic model over entry-observable features only — missing-field indicators, empirical-hull margin, temporal drift, and provenance ambiguity — standardized on the development split's own moments so no calibration statistic enters the fit. Its positive class is an *unproductive* development group: one the artifact does not carry cleanly to a verified result, which on $\mathcal{D}$ means a guard rejection, a verifier abstention, or a manifest mismatch. Wrong

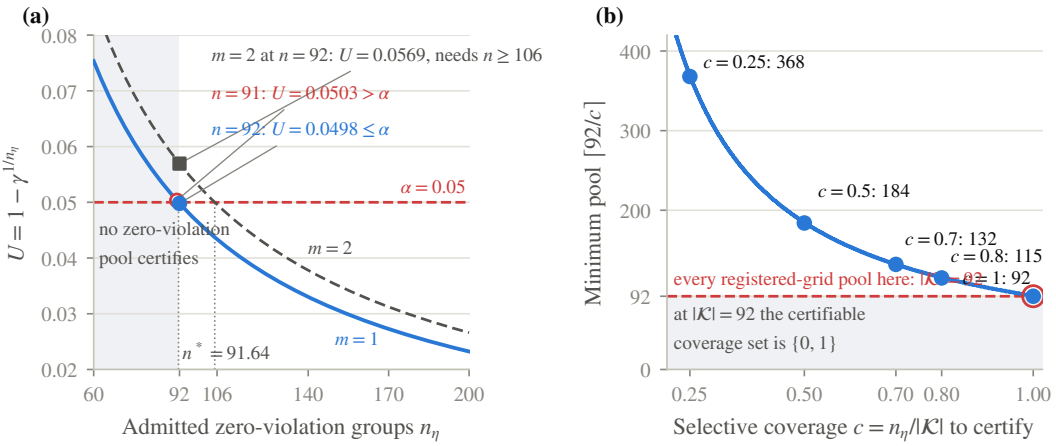


Figure 4: Why a 92-group pool certifies only full coverage. (a): the zero-violation bound $U(n) = 1-(\delta/|\Lambda|)^{1/n}$ at $\delta = 0.1$, $|\Lambda| = 11$ crosses $\alpha = 0.05$ between $n = 91$ (0.0503) and $n = 92$ (0.0498). (b): the smallest pool $\lceil 92/c \rceil$ that can certify selective coverage c; every registered-grid calibration pool in this paper is 92 (the post-cutoff certificate uses 132 groups under a single rule), so the certifiable coverages are 0 and 1 whatever q ranks.

executions are not available as training signal at this point in Alg. 2, because ReplayAndChallenge at line 12 retires any candidate that produces one on $\mathcal{D}$; every candidate reaching FitScore at line 13 therefore has zero contract violations on the development split by construction. This is a design consequence rather than an oversight — the cascade is ordered so that deterministic checks reject before the statistical stage runs — but it does mean q is fitted on abstention-shaped supervision and asked at calibration to rank a different event, which is one plausible reason it discriminates weakly (Section 7).

What cohort size permits the gate to certify. The zero-violation floor $n_\eta \geq 92$ constrains coverage, not just admission. Certifying selective coverage $c = n_\eta/|\mathcal{K}|$ requires $|\mathcal{K}| \geq \lceil 92/c \rceil$: 92 groups for $c = 1$, 115 for $c = 0.8$, 132 for $c = 0.7$, 184 for $c = 0.5$. Every registered-grid cohort in this paper fixes $|\mathcal{K}| = 92$ (the post-cutoff certificate's 132 groups use a single rule), which is the smallest pool at which Eq. (7) is satisfiable at all. At that size the admissible coverage set is exactly $\{0, 1\}$: abstaining on one group leaves $n_\eta = 91$ and $U_\eta = 1-(0.1/11)^{1/91} = 0.0503 > \alpha$. The step-like gate profiles reported in Table 23 and Appendix H are therefore consistent with two different worlds — a score that cannot rank, and a cohort too small to certify a ranking — and these experiments do not separate them. Two intermediate coverages were *observed* and neither is certifiable: `huggingface/datasets` accepts 10/92 groups at $\eta \leq 0.08$ in the gate-frontier study ($U = 0.375$), and `pandas-dev/pandas` accepts 90/92 at $0.08 \leq \eta \leq 0.20$ in the balanced rerun ($U = 0.0509$). A frontier study must first buy calibration groups (Fig. 4). The implementation also supports a minimum-distinct-days requirement, which the single-snapshot live study does not exercise (`min_days = 1`). Two terms of Eq. (5) are coarser in the implementation than the notation suggests: ρ_{eff} is approximated by the fraction of tool names containing a namespace separator rather than read from the effect catalog, and $|F|$ is substituted by the argument-slot count of the first observed window because ranking precedes synthesis. Both affect candidate ranking and thus which candidate is tested, but neither enters the risk bound. The effect catalog is checked during qualification and in the runtime facade.

The two executable-gold external substrates are provider-free after their pinned sources are prepared. Their exact compiler entry points are:

```
B=paper/scripts/bfcl_compiler_benchmark.py
python "$B" --source-root "$BENCHMARK_SOURCE_ROOT"
A=paper/scripts/appworld_compiler_benchmark.py
python "$A" --source-root "$BENCHMARK_SOURCE_ROOT" \
  --appworld-python "$APPWORLD_VENV/bin/python"
D=paper/scripts/appworld_dispatch_preflight.py
```

Table 7: Absolute per-record resources behind the percentages of Table 1: mean / median over the 30 held-out records of each family and over all 90. Estimated dollars are list-price estimates (Appendix C); the entire 90-record compiled evaluation cost an estimated $0.028 against $0.068 for the baseline. Cells are mean / median.

Family	Arm	Requests	Interfaces	Tokens	Latency (ms)	Cost (USD$\times 10^{-4}$)
Issue-type routing	Baseline	4.00	3.00	4,259.4 / 4,320	6,160 / 5,402	8.72 / 8.74
	Compiled	2.00	3.00	2,576.5 / 2,602	2,973 / 2,881	5.93 / 5.74
	Manual	2.00	1.00	1,780.8 / 1,734	3,251 / 2,856	5.45 / 5.49
PR-outcome audit	Baseline	4.00	3.00	2,718.2 / 2,676	5,329 / 4,839	6.79 / 6.66
	Compiled	1.00	1.00	521.5 / 462	1,440 / 1,297	1.68 / 1.51
	Manual	1.00	1.00	522.2 / 458	1,827 / 1,364	1.69 / 1.51
Backlog-attention routing	Baseline	3.97	2.97	2,844.9 / 2,816	6,428 / 5,163	7.29 / 7.10
	Compiled	1.00	1.00	529.9 / 468	2,000 / 1,557	1.82 / 1.62
	Manual	1.00	1.00	535.5 / 479	1,695 / 1,501	1.88 / 1.70
All (n = 90)	Baseline	3.99	2.99	3,274.2 / 2,907	5,972 / 5,163	7.60 / 7.42
	Compiled	1.33	1.67	1,209.3 / 615	2,137 / 1,957	3.14 / 2.05
	Manual	1.33	1.00	946.2 / 616	2,258 / 1,913	3.01 / 2.18

Table 8: Per-record paired differences (compiled minus baseline) for all three families, with 10,000-sample bootstrap intervals (seed 20260802) and two-sided Wilcoxon signed-rank tests over $n = 30$ paired held-out records (exact where differences are tie-free, tie-corrected normal approximation otherwise). Latency is reported as the mean and as the median paired difference, each with its interval; cost intervals are in USD. Provider-request differences are deterministic by construction (every pair removes the same number of turns), so no rank test is reported for them: the statistic would measure the degeneracy rather than the effect. The backlog baseline's one three-request record (5189) is the only exception and appears in Table 9.

Family	Endpoint	Paired difference (95% CI)	p
Issue-type	Total tokens	$-1,682.9$ $[-1,772.5, -1,596.7]$	1.9×10^{-9}
	Wall latency (mean)	$-3,187$ ms $[-3,758, -2,660]$	1.9×10^{-9}
	Wall latency (median)	$-2,716$ ms $[-3,532, -2,309]$	—
	Estimated cost	$-\$2.79 \times 10^{-4}$ $[-3.13, -2.41] \times 10^{-4}$	3.7×10^{-9}
PR-outcome	Total tokens	$-2,196.7$ $[-2,211.1, -2,182.5]$	1.7×10^{-6}
	Wall latency (mean)	$-3,889$ ms $[-4,499, -3,346]$	1.9×10^{-9}
	Wall latency (median)	$-3,476$ ms $[-3,736, -3,245]$	—
	Estimated cost	$-\$5.11 \times 10^{-4}$ $[-5.22, -5.02] \times 10^{-4}$	1.7×10^{-6}
Backlog-attention	Total tokens	$-2,315.0$ $[-2,366.0, -2,224.1]$	1.7×10^{-6}
	Wall latency (mean)	$-4,428$ ms $[-5,400, -3,558]$	1.9×10^{-9}
	Wall latency (median)	$-3,619$ ms $[-4,201, -3,091]$	—
	Estimated cost	$-\$5.47 \times 10^{-4}$ $[-5.66, -5.23] \times 10^{-4}$	1.7×10^{-6}

```
python "$D" --source-root "$BENCHMARK_SOURCE_ROOT"
```

The AppWorld environment uses Python 3.12 with package version `0.1.3.post1` and data version `0.1.0`; the signed source manifest and requirements file pin the remaining acquisition inputs. The BFCL source is pinned to revision `6ea57973c7a6`. These commands regenerate only aggregate, non-sensitive evidence; no model or provider call is made.

D Paired Statistics for the Primary Families

Latency validity. Table 8 gives the paired differences behind Table 1. Latency is wall-clock provider evidence and is secondary to requests and tokens. In the PR-outcome and backlog families each held-out record ran its three arms one at a time on one host, cycling through all six arm orders (five records per order), with no warm-up, a 120 s per-episode timeout, and no retries; the issue-type family used a blocked six-permutation Latin design with eight-way concurrency within each block. No server-side state is shared between arms (`store=False`); cached-input tokens are zero on every PR-outcome and backlog record and are priced separately where they occur

Table 9: Exact-contract discordance per family and pooled. "Baseline-only" counts records only the unchanged agent missed; "compiled-only" records only the compiled arm missed. The upper bound is the one-sided 95% Clopper–Pearson bound on the compiled-only count (0/90 pooled).

Family	n	Both pass	Baseline-only	Compiled-only	McNemar p	Upper bound
Issue-type	30	30	0	0	1.00	9.5%
PR-outcome	30	30	0	0	1.00	9.5%
Backlog-attention	30	29	1	0	1.00	9.5%
Pooled	90	89	1	0	1.00	3.3%

Table 10: Percentage reductions of Table 1 with 95% bootstrap intervals on the ratio of paired sums (10,000 record resamples, seed 20260804). Per-family request reductions are structural (backlog's one three-request baseline record aside); the intervals on tokens and cost are within five points of the estimate, latency within ten. Discovery cost alone breaks even after 411, 183, and 181 compiled episodes per family and per snapshot (`paper/results/cache_accounting.json`); the issue-type baseline's higher cached-input share is why its cost reduction trails its token reduction.

Family	Requests	Tokens	Latency	Cost
Issue-type routing	50.0 [50.0, 50.0]	39.5 [38.7, 40.3]	51.7 [46.7, 56.4]	32.0 [28.3, 35.3]
PR-outcome audit	75.0 [75.0, 75.0]	80.8 [79.1, 82.5]	73.0 [68.2, 77.1]	75.3 [73.3, 77.2]
Backlog-attention routing	74.8 [74.4, 75.0]	81.4 [79.5, 82.9]	68.9 [61.4, 75.3]	75.1 [73.0, 76.9]
Pooled (n = 90)	66.6 [64.1, 68.9]	63.1 [58.7, 67.8]	64.2 [60.4, 67.9]	58.7 [54.1, 63.3]

(`paper/results/cache_accounting.json`). Recomputing the compiled-versus-baseline wall-latency reduction within each of the six arm orders gives reductions between 43.2% and 78.6% across the eighteen permutation–family cells (every cell positive), and provider-response latency reproduces the wall-latency reductions to within 0.3 percentage points, showing consistency across order blocks while not excluding service variance.

Table 11: The hand-written comparator priced on the same resource axes as Table 1, as percentage reduction against the same baseline and the same held-out records. *Manual* is written with full knowledge of the workflow, so this is the pessimistic comparison for the compiler: it ties on provider requests, issues fewer tool interfaces, and is marginally cheaper in tokens and dollars, while compilation leads only on wall latency (bold). The interface gap is concentrated in issue-type routing, where the compiled prefix covers two reads but the agent still issues the third. We therefore claim automated discovery and guarded admission, not runtime dominance over code written by someone who already knows the workflow.

Condition	Requests	Interfaces	Tokens	Latency	Cost
Issue-type routing					
Compiled	50.0	0.0	39.5	**51.7**	32.0
Manual	50.0	66.7	58.2	47.2	37.5
PR-outcome audit					
Compiled	75.0	66.7	80.8	**73.0**	75.3
Manual	75.0	66.7	80.8	65.7	75.2
Backlog-attention routing					
Compiled	74.8	66.3	81.4	68.9	75.1
Manual	74.8	66.3	81.2	**73.6**	74.2
Weighted total (n = 90)					
Compiled	66.6	44.2	63.1	**64.2**	58.7
Manual	66.6	66.5	71.1	62.2	60.4

An extended issue-type held-out cohort. A pre-registered extension (`paper/supplementary/issue-type-extended-heldout-protocol.md`) ran the retained issue-type artifact on 120 further unused records of the pinned snapshot on the calibrated model, balanced 40/40/40 over bug,

Table 12: Extended issue-type held-out cohort on the calibrated model: per-record means over 120 unused records with a disclosed class mix (40 `bug` / 40 `enhancement` / 40 `other`). The compiled-only miss (record 2737) and the pooled bound are the pre-registered primary endpoints.

Arm	Exact	Requests	Tokens	Latency (s)	Cost (¢)
Unchanged agent	118/120	4.00	4,381	6.70	0.090
Compiled two-read prefix (GAC)	118/120	2.00	2,649	3.85	0.061
Hand-written bundle	117/120	2.00	1,813	3.68	0.055
Compiled-only failures: 1/120 here (one-sided 95% bound 3.9%); 1/210 pooled with the primary cohorts (bound 2.2%)					

Table 13: Primary GitHub benchmark at a glance. Each sample supplies one record number from a pinned public snapshot; every listed answer field is checked against the source record.

Task	Read-only evidence path	Exact answer contract
Issue type	`issue_number` → record, official labels, comments	Title, state, label-derived category, verbatim excerpt
PR outcome	`pr_number` → PR record, merge status, discussion	`open`, `merged`, or `closed_unmerged`, plus title and comment excerpt
Backlog attention	`issue_number` → open-issue record, assignees, discussion	`owned`, `discussed_unowned`, or `awaiting_first_response`, plus title, owner, and excerpt

`enhancement`, and `other` (the snapshot holds 16 exclusive `question` issues, 10 already used, so the retained three-way mix could not be reproduced), with the three retained arms in a balanced Latin order and the artifact recompiled provider-free under the current tracer pin (identical id, program, splits, and 92/0 gate). By the decision rule fixed in advance the compiled-only miss comes first: record 2737, where the compiled arm quoted an error line with backticks the source does not contain; the baseline missed record 3040 (a doubled space) and all three arms missed record 3968. Exact contracts were 118/120 baseline, 118/120 compiled, and 117/120 hand-written (Table 12); the artifact dispatched on 119/120 records and abstained cleanly on record 5102 on the induced verifier's cardinality clause, running the unchanged agent. The one-sided 95% bound on compiled-only failures is 3.9% on these 120 records and 2.2% pooled with the 90 primary records (1/210); the pre-registered targets of zero misses, 120/120 dispatch, and a pooled bound at or below 1.5% were all missed and are reported as such. Five of the 120 records had appeared in the gate-frontier pilot's cohort because the exclusion scan omitted that directory; none of the missed or abstained records is among them, and without them the counts are 113/113/112 of 115 with bounds of 4.1% and 2.3%. Requests fell 50.0%, tokens 39.5%, latency 42.5%, and cost 32.8% against the same-model baseline, matching the primary family within a point on requests, tokens, and cost (latency 42.5% against 51.7%). In all, 810 held-out GitHub episodes ran under a compiled artifact on the calibrated model in six studies: 270 on the excerpt-bearing contracts (the 90 primary, these 120, and the 60 post-cutoff records of Appendix G), with this cohort's record 2737 the one compiled-only miss (one-sided bound 1.7%), and 540 on the outcome-only cross-repository contract (120 core, 180 balanced, and 240 gate-frontier episodes over 354 distinct records; Appendices F and H), with none.

E Additional Experimental Detail

Primary workflow families. The three-family result is the main paper's strongest live evidence because it combines distinct tool vocabularies, exact task graders, balanced held-out cohorts, and provider-backed execution. The compiler limits the issue-type family to a shallower two-read prefix; the newer pull-request and backlog families admit deeper pre-model compaction. This difference explains the lower resource savings on the first family and is part of the evidence boundary, not noise to average away.

Cross-repository extension. The cross-repository study is narrower than the primary workflow families. Its task uses two read-only tools and a simpler exact outcome contract, which is precisely

Table 14: Held-out dispatch in the cross-repository studies. "Dispatched" counts records on which the compiled program ran; the remainder abstained on the named verifier clause and ran the unchanged agent, which is why the request reduction is 44.4% under the core protocol and 66.7% under the balanced one for the same two-read artifact.

Protocol	Repository	Held-out	Dispatched	Fallback clause	Req. ↓
Core	`huggingface/datasets`	30	20/30	`range:pr.state` (10)	44.4%
Core	`pandas-dev/pandas`	30	20/30	`range:pr.state` (10)	44.4%
Core	`psf/requests`	30	20/30	`range:pr.state` (10)	44.4%
Core	`pytorch/pytorch`	—	RETIRE	—	—
Core	`streamlit/streamlit`	30	20/30	`range:pr.state` (10)	44.4%
Balanced	`pandas-dev/pandas`	60	60/60	—	66.7%
Balanced	`psf/requests`	60	60/60	—	66.7%
Balanced	`pytorch/pytorch`	60	60/60	—	66.7%

why a fixed template can remain tied or even more efficient than the learned artifact. We keep the result in the main text because it addresses a central scope question — whether guarded specialization survives beyond one repository — while being explicit about the reduced task complexity.

The position invariant, empirically. An early pilot dispatched a compiled *suffix* region at the entry boundary, violating Eq. (2). Because the runtime resolves artifacts before the agent has made the calls the region assumed had already happened, the pilot reordered and duplicated tool calls. The failure is a semantic mismatch between compilation-time position and resolution-time position, not a tuning error, which is why Eq. (2) is a hard constraint in the deployed compiler rather than a scoring penalty.

Omitted from the main claim. The repository retains additional studies that are useful scientifically but less suitable for the main narrative:

- a natural-order issue study that exposed a factual continuation miss for a more aggressive three-read artifact,
- exploratory guarded composite synthesis and a follow-up comparison to a fair pre-model manual program,
- a bounded GEPA prompt-optimization study in which GEPA retained its seed, so the combined arm is a replication rather than evidence of interaction, and
- a portfolio pilot and controlled demonstration suite for runtime boundary coverage, including workloads where removing turns *increases* cost through prefix-cache fragmentation.

These studies remain important to the broader artifact, but they either use a weaker risk level, address a narrower engineering question, or are explicitly exploratory.

F Evidence Breadth and Boundary Audits

The main text covers the registered live GitHub studies, the cross-repository extension, the BIRD live study, and the NESTFUL and AppWorld substrates. This appendix makes the retained breadth visible without pooling incomparable cohorts or promoting screened and exploratory work to deployment evidence.

F.1 Repository-level outcomes on newer records

The core `pytorch/pytorch` run mines one candidate from 116 accepted windows and retires at calibration. A provider-free recompilation (`paper/results/iclr_revision/pytorch_core_recompile.json`) recovers $n_\eta = 0$ at every threshold, identifying the empty-sample branch of Eq. (7). Discovery contains 84 closed-unmerged, 30 open, and 2 merged records; the held-out split is balanced at 10 per class. The balanced rerun admits the same task. These observations are consistent with sensitivity to cohort composition, but neither the degenerate bound nor the rerun establishes the cause of the original refusal.

The core result succeeds on four repositories (120/120 held-out pairs) and retains the fifth as a compile-time retirement after 116 exact discovery traces. The balanced rerun has a distinct three-

Table 15: Disaggregated cross-repository outcomes behind Section 5.2. The core protocol uses four completed repositories and one compile-time retirement; the balanced rerun changes the selected repositories and class mix, so it is reported separately rather than pooled with the core protocol. Reductions compare the compiled condition with the paired unchanged baseline.

Protocol	Repository	Held-out	Exact	Req. ↓	Tokens ↓
Core	huggingface/datasets	30	30/30	44.4%	52.2%
Core	pandas-dev/pandas	30	30/30	44.4%	52.5%
Core	psf/requests	30	30/30	44.4%	52.3%
Core	streamlit/streamlit	30	30/30	44.4%	52.7%
Core	pytorch/pytorch	—	RETIRE	—	—
Balanced	pandas-dev/pandas	60	60/60	66.7%	78.6%
Balanced	psf/requests	60	60/60	66.7%	78.7%
Balanced	pytorch/pytorch	60	60/60	66.7%	78.0%

Table 16: All four trace benchmarks used as compiler substrates. NESTFUL and AppWorld are discussed in Section 5.4; API-Bank and executed BFCL v4 gold plans retire on support exactly like NESTFUL. Columns as in Table 2.

Substrate	Traces	Slots (1)	Windows (2)	At floor (2)	Synth. (4)	Replay p/a/**w** (5)	$n_{\max}$ (6)	Adm.
NESTFUL	1,415	96.3%	1,207	32	12	24 / 12 / **0**	26	**0**
API-Bank	212	63.5%	37	8	2	0 / 2 / **0**	8	**0**
BFCL v4	200	68.0%	86	9	4	3 / 3 / **0**	15	**0**
AppWorld	136	63.2%	136	33	1	34 / 0 / **0**	**136**	**1**

repository cohort and reaches 180/180 held-out pairs. These rows show breadth across repositories drawn from shared pools — the balanced rerun shares 13, 25, and 8 of the 30-record core held-out sets — not independent evidence for an average effect: all runs share the same provider family and the same narrow two-read outcome task.

F.2 Continuation audit

The earlier natural-order three-read study is excluded from the registered $\alpha = 0.05$ headline because it used $\alpha = 0.10$ and exposed a downstream continuation error. Its retained counterfactual replay nevertheless tests a boundary the exact tool contract cannot see: 17/18 candidate continuations pass unchanged, while issue 6602 has a comment-evidence mismatch. Checked rendering repairs that one case, yielding 18/18 final contracts. This audit makes no provider calls and does not estimate live agent quality; it establishes only that the continuation check caught a concrete failure the compacted tool trace alone would miss.

F.3 External evidence map

Table 18 records every named external benchmark in the retained evidence register and its admissible interpretation.

Obtaining observed results from BFCL. BFCL's reference plans carry no tool results, which is why the plan-format row and the compiler row are listed separately. Its multi-turn categories, however, ship an executable stateful backend: each task publishes an `initial_config` snapshot and the classes it involves, so executing the *official gold plan* through the upstream helper yields observed results, an entry snapshot, and an in-process replay oracle, with no model in the loop. Five preconditions fail the run closed rather than publishing: an undeclared method, any upstream execution error, a read-like declaration observed mutating state, a compilable declaration observed advancing the scenario RNG, and any inexact re-execution. All held: 200 tasks and 1,142 calls executed cleanly and a second independent pass reproduced every result byte for byte, executing turn-at-a-time where the first pass executed call-at-a-time.

Table 17: Structural dispatch eligibility of the admitted AppWorld artifact on the 28 released official baseline runs. Eligibility is the admitted program's exact call sequence at normalized position 0; it is an upper bound on φ, because the manifest check and the calibrated gate are not evaluated and the released logs retain API calls rather than model boundaries. Trajectories repeat tasks across runs and are descriptive rather than independent samples. A miss abstains to the unchanged agent.

Released baseline architecture	Runs	Trajectories	Eligible at pos. 0	Rate
Full code + reflection	8	2,340	2,339	1.000
Iterative parallel function calling	4	1,170	762	0.651
Plan and execute	8	2,340	1	0.000
ReAct	8	2,340	0	0.000
Pooled	28	8,190	3,102	0.379

Why the catalog is signed rather than inferred. The same run audits the declarations empirically by snapshotting every involved instance around every call. It observed 1,060 state-mutating calls over 44 of the 81 methods and 94 calls that advanced the seeded scenario RNG, with zero cases of a read-like declaration mutating state or a compilable declaration advancing the RNG. One case is worth naming: a method whose name and docstring present it as a cost getter mutated its instance's internal lookup on all 36 of its calls and is therefore declared a write. A name- or docstring-derived effect label would have admitted a write into a compiled region. Two further methods are read-like but carry no capabilities, because one derives an age from the host date and the other draws from — and advances — the seeded RNG; they are barriers by declaration and account for 35 suppressed spans. Write barriers suppress 2,802 spans in total, against 45 for entry-schema admissibility, 31 for slot ambiguity, and 5 for partial runs.

Obtaining observed results from AppWorld, and why it is the reachable case. AppWorld's shipped `api_calls.json` records method, url, and request data but no responses, so it fails closed for the same reason BFCL's plans do. Executing the official `compiled_solution_code` against the pinned `0.1.3.post1` backend supplies the results; each task's own `specs.json` supplies the entry snapshot; a second execution supplies the replay oracle. Calls are captured at the single dispatch point every `apis.<app>.<api>(...)` call passes through, so the recorded identity is a semantic app/API pair with typed keyword arguments and a decoded JSON result. Of the 147 public train+dev gold solutions, 136 execute cleanly; 11 dev solutions raise against the shipped data version and are reported as upstream compatibility outcomes rather than dropped. The second pass reproduced 7,070/7,070 results byte for byte, minted access tokens included, because AppWorld freezes each task's clock.

The empirical audit attributes every `INSERT/UPDATE/DELETE/CREATE/DROP` statement to the enclosing call. It observed 1,320 mutating calls, zero read-like declarations mutating, and — unlike BFCL — zero APIs that mutate on some calls but not others. Every `GET` was clean on every call and every `DELETE` and `PATCH` mutated on every call, so the effect boundary is crisp; the five `*.login` endpoints are the only non-`GET` APIs that never mutate, which is why they are the single contestable declaration and why the run is pre-registered in two arms. Groups are tasks, not scenarios: AppWorld's three variants of a scenario share a supervisor and a database, so the bound is conditional on task-level independence, which this corpus does not establish — the same conditional the live studies carry, neither weakened nor repaired here; at scenario or supervisor level (46 and 79 units) the zero-violation bound is 0.097 and 0.058, both above α. Arm A calibrated one candidate and is compiler-wide; arm B calibrated six (four retired, one dominated, one emitted), so its certificate is per-candidate, with a corrected bound of 0.068. The calibration share is fixed at exactly the 92 groups the bound needs, since a larger set would make the bound easier and a smaller one would make it unreachable. Two held-out replay figures appear for AppWorld and they count different partitions, which Table 2 flags but does not have room to explain. The 34/0/0 in that table is the test side of the *family* partition used to challenge a mined candidate — 102 train and 34 test groups, the same accounting every substrate row reports. The separately reported 11/11 admission-test passes are from the sealed test split of the 11/92/11 dev/calibration/test partition Alg. 1 consumes for admission. Both are genuinely held out; they are held out from different fits, and neither is a subset of the other's training side.

Table 18: Disposition of every retained external benchmark family. Only the four compiler-measurement rows and the BIRD live study support compiler claims; the remaining rows document why broader benchmark names are not silently converted into GAC results. BFCL appears twice because its reference plans and its executed gold plans are different evidence.

Benchmark	Evidence tier	Retained result / claim boundary
NESTFUL	Compiler measurement	1,415 traces; 24 / 12 / 0 replay; all families retire.
API-Bank	Compiler measurement	212 traces; 0 / 2 / 0 replay; all candidates retire.
BFCL v4 (plans)	Gold-plan checker	200/200 valid plans; plan format only.
BFCL v4 (executed)	Compiler measurement	200 traces, 1,142/1,142 exact re-execution; 3 / 3 / 0 replay; all families retire.
AppWorld (executed)	Compiler measurement	136/147 traces, 7,070/7,070 exact re-execution; $n_{\max} = 136 \geq 92$; one artifact admitted at $U = 0.0498$; under arm B two candidates retire at the margin ($U = 0.051$) and two far from it ($U = 1$).
BIRD (dev and train)	Live-provider compiler study	31 databases, two agent designs, three models: the standard design retires everywhere; the schema-first opening is admitted on 26 of 31 databases (`gpt-5.6-luna`) and 5 of 5 (`gpt-6-luna`); no detected accuracy loss on 924 held-out questions (Appendix F.4).
ToolSandbox	Live-provider subset	One simulated scenario; compiler not run.
τ^2/τ^3	Live-provider subset	0/4 pass; compiler not run.
ToolBench	Adapter smoke	Ten fixtures; full data/backend unavailable.
AgentBench	Adapter/preflight	Services and Freebase data gated.
GAIA	Access preflight	Authorization denied; no metric.
SWE-bench Verified	Task adapter	Official run host-gated.
BrowseComp	Live-web subset	1/3 correct, 28 searches; bypassed by the compiler.

Admissible is not dispatchable. Table 17 separates the two. The admitted region is eligible on essentially every full-code trajectory and essentially no ReAct or plan-and-execute trajectory, because those architectures open by reading API documentation—3,752 of the 8,190 released trajectories begin with an API-description call — so the compiled prefix is no longer at position 0 and (2) refuses. A further 61 trajectories contain the program somewhere other than position 0, which the same invariant refuses. Full-code agents are eligible on 2,339/2,340 trajectories and iterative parallel function-calling agents on 762/1,170. Two secondary observations: the rate is flat across task difficulty (0.377 on `test_normal` against 0.380 on `test_challenge`), consistent with the strong association with agent architecture, without excluding workload effects; and the maximal compilable depth is 2 on the eligible families, so the admitted two-step program sits at the ceiling the effect catalog permits rather than short of it. On the ineligible families the measured depth is a floor, not an estimate, because the API-documentation call is undeclared in the signed catalog and unknown effects are not reads.

F.4 A live agent on a public benchmark: BIRD

Design. The study was registered before any provider call (`paper/supplementary/bird-sql-agent-protocol.md`). A tool-using SQL agent on `gpt-5.6-luna`, with the paper's pinned settings, answers BIRD dev questions (Li et al., 2023a) (release `dev_20240627`, question plus evidence) over read-only SQLite files with three tools: `list_tables`, `get_schema(table_names)` with three sample rows, and `run_query`. The contract is BIRD execution accuracy: the final query's result set equals the gold query's. Every dev database with at least 146 questions (16 train, 8 dev, 92 calibration, 30 held out) is a family, which selects five. Two agent designs were registered: *standard*, adapted from LangChain's SQL-agent prompt (LangChain, 2026) ("query the schema of the most relevant tables"), and *schema-first*, whose workflow step reads every table before writing a query, the analogue of the paper's prescribed design. QUALIFY does not filter on answer correctness, the compiler runs with the registered settings, and a strict audit vetoes any admission whose calibration groups show the agent passing different opening arguments than the program derives.

Table 19: BIRD: compile-or-retire per database and design, and the schema-first held-out evaluation (30 sealed questions per database). "Repeat" is a second independent run of the unchanged agent. Reductions compare the compiled agent with that warm-cache repeat run (the condition order was not rotated as registered; see text). Standard-design retirements are at synthesis (`ungroundable_slot`) or on calibration support.

		Standard	Schema-first: correct of 30				Compiled vs. warm repeat		
Database	Tables	GAC	Base	Repeat	Compiled	Manual	Requests	Tokens	Cost
`card_games`	6	retire: support 34/92	18	16	**19**	18	44.1%	7.8%	−2.6%
`codebase_community`	8	retire: synthesis	22	23	**22**	21	48.8%	15.2%	6.3%
`formula_1`	13	retire: synthesis	18	19	**20**	19	43.7%	4.2%	1.8%
`student_club`	8	retire: synthesis	26	25	**24**	24	49.6%	19.0%	18.7%
`thrombosis_prediction`	3	retire: synthesis	10	11	**12**	12	39.0%	6.0%	12.1%
Pooled		5 retire	94	94	**97**	94	44.9%	9.8%	6.3%

Admission. The standard agent passed 7 to 39 distinct table lists per database and every table in no discovery run on four of them, so `get_schema.table_names` has no consistent expression and four families retire at synthesis (Table 19). On `card_games` the candidate "read only `cards`" (the agent's most common choice, 61 of 132 runs) matched 34 calibration groups and retired on support. In 7 of those groups the agent had read a different table; the compiler labeled them unproductive, which stays in the count, rather than violations. That is the leniency the strict audit was registered to catch, and with more support the compiler alone could have admitted a wrong-shaped program. The schema-first agent passed the full listing in every discovery run, and every family admitted `list_tables` → `get_schema`(all tables) with one candidate at calibration (compiler-wide), 92 groups, zero violations, and a clean audit. The table list is bound as an invariant literal, not as an expression over the listing as the protocol predicted, because the implemented rule tries literals first.

Held-out evaluation. The compiled program dispatched on 150 of 150 sealed questions. Execution accuracy was 97/150 compiled, 94/150 for the unchanged agent, 94/150 for its repeat, and 94/150 for a hand-written schema prefetch; compiled against the unchanged agent there are 5 compiled-only and 2 baseline-only correct answers (McNemar $p = 0.45$; difference +2.0 points, 95% interval $[-1.3, +5.3]$), which meets the registered criterion for no detected accuracy loss. The interval also permits a loss; neither equivalence nor noninferiority was established. The registered rotation of condition order was not implemented: the four conditions ran in a fixed order, so the first baseline met a colder provider prompt cache (cached-input share 0.504 against 0.56–0.59) and its repeat was 11.9% cheaper with the same tokens. Against the warm repeat, the compiled agent made 44.9% fewer model requests $[42.1, 47.4]$, used 9.8% fewer tokens $[4.7, 14.3]$, ran 29.4% faster $[23.6, 35.1]$, and cost 6.3% less $[1.5, 10.6]$; against the first run the cost reduction is 17.5%, and pricing every input token at the uncached rate gives 11–12% either way. The hand-written prefetch reaches 46.3%, 12.4%, 31.5%, and 7.9%, so the discovered program matches the manual ceiling. Savings in tokens and cost are modest because the two removed calls carry the shortest contexts. Total spend was $2.69.

Extensions: order, models, providers, and scale. Four extensions were registered before any of their provider calls (`paper/supplementary/bird-sql-agent-extensions-protocol.md`; Table 20). A rerun with the condition order rotated as originally registered reproduces the result on the same 150 questions (100 against 94 correct; McNemar $p = 0.07$, no superiority claimed) with 46.0% fewer requests and 16.2% lower cost against the first baseline. On a second model family, `gpt-6-luna`, the standard design retires on all five databases and the schema-first opening is admitted on all five, with 49.9% fewer requests and 93 against 91 correct on 149 complete-case questions (150 were scheduled). On the BIRD training split (26 databases with at least 146 questions, after a pre-execution rule excluded one whose gold SQL references tables missing from the shipped database), the standard design retires on all 26 and the schema-first opening is admitted on 21; the five refusals are databases (11 to 65 tables) where the agent occasionally dropped or reordered a table, four on calibration support (90 or 91 of 92 groups) and one at synthesis. On the 625 four-arm-complete questions across the 21 admitted databases the compiled agent dispatched every time and answered 400 correctly against 401 ($p = 1.0$), with 45.9% fewer requests, 12.6% fewer tokens, 29.3% lower latency, and 18.1% lower cost. Pricing every input token at the uncached rate, which removes provider-cache effects, gives cost reductions of 11–18% in every run. On a second provider,

Table 20: BIRD runs. "Std." and "S.-first" count databases admitted per design (the rotated rerun reuses the primary artifacts). Correct is on the held-out questions completed by all four conditions; bold marks the compiled arm, not a significant improvement. Reductions compare the compiled agent with the unchanged agent; †every primary-row reduction is against the warm repeat run because that run's order was not rotated. Cost* prices every input token at the uncached rate.

Run	Model	DBs	Std.	S.-first	n	Correct (base / GAC)	Requests	Tokens	Latency	Cost	Cost*
Primary (fixed order)†	`gpt-5.6-luna`	5	0	5	150	94 / **97**	44.9%	9.8%	29.4%	6.3%	11.2%
E1 rotated rerun	`gpt-5.6-luna`	5	—	—	150	94 / **100**	46.0%	12.3%	33.9%	16.2%	13.9%
E2 second model	`gpt-6-luna`	5	0	5	149	91 / **93**	49.9%	16.8%	38.4%	6.1%	17.5%
E3 second provider	`claude-sonnet-5`	5	0	0	—	—	—	—	—	—	—
E4 train split	`gpt-5.6-luna`	26	0	21	625	401 / **400**	45.9%	12.6%	29.3%	18.1%	14.0%

Table 21: Retrospective BIRD quality over all scheduled questions. Missing runs count as incorrect in their own arm; "Complete" is the historical four-arm intersection. Rotated repeats the primary questions.

Run	Scheduled	Complete	Base	Repeat	GAC	Manual
Primary	150	150	94	94	97	94
Rotated	150	150	94	93	100	96
Second model	150	149	91	95	93	90
Train split	630	625	403	412	402	404

Anthropic `claude-sonnet-5`, the agent followed neither opening as a fixed step: under the schema-first prompt it listed the tables first in 429 of 655 runs and passed every table in 26, and GAC retired all ten (design, database) pairs. GAC compiles what an agent does, not what its prompt asks for. Extension spend was $43.93.

Attrition and all-scheduled denominators. The historical summaries above retain only questions completed by all four arms. This excludes one second-model question because the repeat baseline timed out, and five training-split questions: four manual-arm timeouts and one compiled-arm turn-limit failure. A retrospective audit of the sealed run records counts every scheduled question, treating each missing run as incorrect only in its own arm (Table 21). Training-split accuracy is 402/630 compiled versus 403/630 unchanged, with 17 compiled-only and 18 baseline-only correct ($p = 1$); second-model accuracy is 93/150 versus 91/150 ($p = 0.73$). Neither result establishes noninferiority. Resource comparisons require completed runs: using all baseline/compiled pairs gives request/cost reductions of 45.8/18.0% on 629 training pairs and 49.6/5.8% on 150 second-model pairs. These estimates omit resources consumed by failed runs; aggregate billed cost and time-to-success are not established by them. The script `paper/scripts/audit_bird_denominators.py` regenerates these counts without provider calls. The pinned BIRD release is `dev_20240627`; these selected-family results are not a leaderboard score on the newer cleaned development set.

How much quality loss remains compatible with the data? The all-scheduled audit supports a retrospective uncertainty analysis (Table 22), without selecting a noninferiority margin after seeing the results. Let p_+ denote the probability of a compiled-only correct answer and p_- a baseline-only correct answer. The paired accuracy difference is $\Delta = p_+ - p_-$. Construct a two-sided 97.5% Clopper–Pearson interval for each probability (Clopper & Pearson, 1934); if their endpoints are $[L_+, U_+]$ and $[L_-, U_-]$, then $[L_+ - U_-, U_+ - L_-]$ covers Δ with at least 95% probability under i.i.d. question pairs, by a union bound. Dependence between the gain and loss indicators within a pair is allowed. These conservative, pointwise intervals are wider than the primary study's paired bootstrap interval; neither construction proves the sampling assumption. They are not simultaneous across the four rows, and overlapping primary, rotated, and second-model questions are not pooled as independent evidence.

The training interval allows a loss of 3.27 points or a gain of 2.96 points. Deleting one database keeps its observed difference between −0.50 and +0.33 points; 5 databases favor the compiled agent, 10 tie, and 6 favor the unchanged agent. The second-model difference changes sign under a database deletion. This tests concentration of the observed result, without treating the selected databases as a random sample or establishing generalization to new databases. The retained script

Table 22: Retrospective BIRD quality sensitivity on all scheduled questions. All differences are compiled minus unchanged, in percentage points. Intervals assume i.i.d. question pairs. "Delete-one" omits each entire database in turn; it is a descriptive range, not a confidence interval. The last column counts databases with positive, zero, or negative differences.

Cohort	Δ (pp)	Conservative 95% interval	Delete-one range	DBs +/0/−
Primary	+2.00	[−4.40, +8.18]	[+0.83, +4.17]	3/1/1
Rotated	+4.00	[−2.53, +10.11]	[+1.67, +5.83]	3/1/1
Second model	+1.33	[−5.44, +7.97]	[−0.83, +3.33]	3/0/2
Train split	−0.16	[−3.27, +2.96]	[−0.50, +0.33]	5/10/6

`paper/scripts/bird_quality_sensitivity.py` regenerates the intervals, database deletions, and source hashes.

Sensitivity to missing execution cost. The same training records retain \$0.891759 of estimated cost for 630 completed baseline runs and \$0.728806 for 629 completed compiled runs. Let $u \geq 0$ be the unrecorded cost of the single failed compiled execution, priced with the same frozen rate card. Net evaluation savings are then \$0.162952 minus u, so they are positive only if $u <$ \$0.162952. This is a break-even condition, not evidence that the unknown cost satisfies it. It includes the failure's quality penalty above but cannot recover its missing usage; discovery, recompilation, and maintenance remain excluded.

F.5 The registered gate is a support threshold

This audit strengthens the evidentiary record precisely by preserving the negative result: the registered gate behaves as, and at $|\mathcal{K}| = 92$ can only behave as, a support threshold rather than a demonstrated risk–coverage frontier (Table 23); the question of q's ranking quality is open, not answered negatively. The exploratory partial frontiers therefore remain diagnostic rather than evidence for the main claim.

G Extended Limitations

Every protocol in this paper is in-repo and author-held; no external registry was used, and each is dated in its own file.

The certified event is not end-to-end quality. Proposition 1 bounds the replay-contract event of Appendix C. On these cohorts that event has no empirical support: the argument pattern of the compiled steps is invariant across all discovery traces per GitHub family, and the labels replay those calls over recorded results of a pinned snapshot, which is why no retained pool records a violation, every gate is step-like, and admission reduces to the 92-group floor (the second-provider backlog refusal turns on 90 and 91 eligible groups). The misses that do occur on these families are continuation misses (an excerpt that departs from its source), and they are outside the certificate: the one-sided 95% upper bound on compiled-only discordance in Appendix D is an empirical quality statement at a different confidence level, not an admission certificate. An end-to-end certificate needs a pre-registered endpoint that runs the continuation and grades the task contract on fresh calibration groups, with the sample floor of Eq. (7) applied to that event; one was obtained after the main study for a re-derived PR-outcome artifact on 132 fresh groups (0 misses, bound 0.0173 at 90% confidence; below), and is reported separately from the primary replay-contract certificates.

Dependence sensitivity. The exact binomial bound requires i.i.d. units. One valid cluster-level formulation uses i.i.d. clusters and the maximum violation within each cluster, targeting risk for a new cluster rather than a new record. Distinct days or authors do not establish that assumption; Table 24 is a hypothetical sensitivity calculation. Temporal spacing alone does not ensure independence, and block bootstrap or dependence-aware conformal methods require their own assumptions and guarantees; they do not inherit Eq. (7). For clean-checkout reproduction, the re-

Table 23: Gate-behavior inventory from the retained threshold analysis. A partial frontier would show graded coverage as the threshold changes; none appears among the seven artifacts in the retained threshold analysis.

Evidence set	Artifacts	Partial frontiers	What it licenses
Registered GitHub $\alpha = 0.05$ (threshold analysis)	7	0	Exact per-candidate certificates; all are step-like.
Exploratory or looser-α	3	2	Diagnostic evidence only; not the registered claim.
NESTFUL refusal	12 synthesized	0	$n_{\max} = 26 < 92$; no admission.
API-Bank refusal	2 synthesized	0	$n_{\max} = 8 < 92$; no admission.
BFCL v4 refusal	4 synthesized	0	$n_{\max} = 15 < 92$; no admission, pre-registered.
AppWorld admission	1 admitted / 4 retired	0	$n_{\max} = 136 \geq 92$; the one reachable external bound. Admitted gate is step-like; two retirements are marginal ($U = 0.051$ at 89 and 90 eligible groups), two are not ($U = 1$).

Table 24: Distinct grouping units in the 92-record calibration cohorts, joined to the pinned snapshot: distinct creation days, months, and authors, and the zero-violation bound the registered budget would give if each cluster were one unit (admits only when the count is at least 92).

Cohort	n	Days	Months	Authors	U at days	U at authors
Issue-type routing	92	90	50	82	0.051	0.056
PR-outcome audit	92	86	45	60	0.053	0.075
Backlog-attention routing	92	87	46	73	0.053	0.062
Core: `huggingface/datasets`	92	80	37	46	0.057	0.097
Core: `pandas-dev/pandas`	92	92	66	53	0.050	0.085
Core: `psf/requests`	92	90	66	79	0.051	0.058
Core: `streamlit/streamlit`	92	87	33	25	0.053	0.171

tained `paper/results/iclr_revision/calibration_cluster_rows.json` records the calibration row identifiers, dates, and hashed author equality keys derived from the digest-checked source snapshots.

A read-only prologue does not recover ReAct dispatch. Section 5.4 shows Eq. (2) refuses essentially every ReAct and plan-and-execute trajectory, which open with an API-documentation read. We built the narrowest relaxation (`paper/supplementary/read-only-prologue-protocol.md`): an artifact may declare an exact, allowlisted, read-only prologue whose calls must equal the committed observations in tool, order, and arguments; the prologue's results become live-ins of the region, its calls are not replayed, and any mismatch, extra call, order change, manifest mismatch, unknown effect, or slot left ungrounded by the extended entry contract falls back. The runtime passes the protocol's acceptance tests, including the archived suffix-dispatch pilot as a regression test. Measured on the 8,190 released trajectories with the documentation call as the prologue (Table 25), it changes little: ReAct eligibility rises from 0 to 2 of 2,340 and plan-and-execute stays at 1. The looser diagnos-

Table 25: Structural dispatch eligibility before and after admitting an exact read-only prologue (the API-documentation call these architectures open with), per released AppWorld baseline architecture; never pooled. The runtime rule and the measurement rule are the same function.

Released baseline architecture	Runs	Trajectories	Before (pos. 0)	Rate	After (pos. 0 or exact prologue)	Rate
Full code + reflection	8	2,340	2,339	1.000	2,339	1.000
Iterative parallel function calling	4	1,170	762	0.651	762	0.651
Plan and execute	8	2,340	1	0.000	1	0.000
ReAct	8	2,340	0	0.000	2	0.001

tic — any documentation-only prefix followed by the admitted program — recovers 2 and 3, and the program occurs anywhere in only 16 and 39 of those trajectories. Position was the proximate reason those architectures were refused; the larger limitation is that they seldom execute the admitted region at all. A prologue rule cannot recover those absent executions. The prologue tool is also undeclared in the signed catalog, and the runtime refuses an undeclared prologue; the measurement assumes the declaration a reviewer would have to sign.

Missing automated workflow-reuse comparison. AWM, plan caching, EvoC2F, and Agent JIT differ in execution placement, verification, and reuse unit (Wang et al., 2025; Zhang et al., 2025; Wei et al., 2026; Winston et al., 2026); LLMCompiler emphasizes within-task scheduling (Kim et al., 2024). Those differences complicate matched implementation but do not preclude a quality–cost comparison. An executable automated-reuse baseline remains missing. Internal ablations measure the cost of the method's own barriers; they do not substitute for that comparison.

Table 27 prices the barriers: for each retained hazard it names the guard that caught it and what deterministic replay of the mined region would have done with provenance, effect, position, contract, and risk barriers disabled. On PR-outcome and backlog routing the mined three-read sequence *is* the admitted program, so an unguarded replay executes the same calls and differs only in guards that never fired on in-distribution held-out records; on issue-type routing the compiler refused the three-read candidate, so its price was measured live under the pre-registered protocol (`paper/supplementary/recurrence-only-ablation-protocol.md`). A barrier-free replay of the refused three-read region — no provenance check, guard, verifier, gate, or position check — handed to one provider request on the same sealed 30 records passed 30/30 exact contracts with one request per record (Table 26); every excerpt was located in its source comment. By the decision rule fixed in advance, on this cohort the provenance refusal cost one request per record without a measured quality benefit, and the guard's value here is the refusal of an unwitnessed argument plus the retained issue-6602 counterexample, not a held-out failure. The measurement concerns one family, one cohort, and in-distribution records; it says nothing about behavior under shift, which the drift protocol below is written to test.

Provider-free drift ablation: the pre-declared null. The drift-robustness protocol (`paper/supplementary/drift-robustness-ablation-protocol.md`; amended before execution and run on 2026-10-06, provider-free) asks whether the induced verifier converts wrong answers into abstentions under the nine declared perturbation families. On the deterministic demonstration substrate it runs two arms over an identical compiled program and identical sealed-test windows, differing only in the verifier (induced versus permissive), with the hard runtime boundary retained in both; the hand-written arm is not run because the demonstration comparators are policy-level macros rather than IR programs. Six admitted artifacts over three demonstrations give 144 paired groups (one window per group). Both arms produce zero wrong outcomes (one-sided 95% upper bound 0.0206 per arm; no discordant pair), so the suite does not separate the arms on this substrate: the simulated tools remain total, and the unverified program's live-outs under nulled fields, duplicated records, padded lists and schema drift equal its unperturbed live-outs. What the verifier does here is abstain, including on invariant perturbations where the answer was unchanged (0.2477 pooled; 0.3333 on every `permissioned_rag` artifact, above the protocol's 0.25 guardrail and below its 0.50 bar), which is a cost without a measured benefit on this substrate. This is a null, not equivalence; it says nothing about the GitHub families or real workloads. Two further observations are reported as such: `incident_triage` admits no artifact under the current compiler although the retained 2026-08 run admitted two, and `mcp_ops` retires in both.

Recorded-replay drift ablation on the primary records: the null, and what it reveals about the oracle. The same question on real records (`paper/supplementary/drift-recorded-replay-protocol.md`; pre-registered and run 2026-10-06, provider-free): the three retained artifacts, recompiled identically from their sealed checkpoints, run with their induced verifier, with a permissive verifier, and beside the family's hand-written pre-model program with a permissive verifier, over the 90 primary held-out records reconstructed on the pinned snapshot (89 paired records; the baseline-only miss record 5189 yields no window of its family). All nine transforms change at least one recorded call per family. All three arms produce zero wrong outcomes (upper bound 0.0331 per arm; no discordant pair; Holm-adjusted exact McNemar $p = 1$ for both comparisons). The reason is structural, not total tools: the suite's oracle is the sequence of derived call arguments, and every admitted GitHub program's only derived argument is the record identifier, bound at entry and re-read unchanged from the first record tool's echo; none of the nine transforms alters that identifier, so no tool-result perturbation in this suite can change a decision and *wrong* is unreachable for these programs. What the suite does measure is the contract: the induced verifier abstains on nulled fields and on duplicated or padded discussion (or label) lists (0.1049 of invariant-family outcomes pooled; 0.0444, 0.1222, 0.1494 per family; padded lists are expected-either and sit outside that denominator), the interpreter abstains on schema drift in both compiled arms, and the hand-written program answers through schema drift unchanged because it echoes what it is given. Those abstentions guard the continuation against malformed evidence, which this endpoint does not grade; a live continuation extension would. Not equivalence; and a program whose later arguments derive from earlier results is where a verifier benefit could appear under this suite.

Post-cutoff cohort and an end-to-end certificate (PR-outcome). After the main study we acquired every `huggingface/datasets` issue and pull request created after the pinned snapshot's last day (2025-06-13; 1,095 records, 1,086 of them passing a preflight that drops any record number a retained result mentions) and pre-registered a study (`paper/supplementary/time-forward-end-to-end-protocol.md`) on the one family whose fresh pool supports it: PR-outcome (issue-type and backlog routing are NO-GO on class availability, as before). From 811 fresh pull requests, three disjoint splits were frozen before any call: 60 test records (20 per class), 132 discovery records and 132 calibration groups. The calibration groups, taken next in rank order after the balanced test draw and the round-robin discovery draw had consumed 64 of the pool's 72 closed-unmerged records, are 84 open and 48 merged pull requests and no closed-unmerged ones; the certified population is therefore fresh open or merged pull requests, and the 20 closed-unmerged test records below are uncertified. The retained artifact's verifier pins the old snapshot revision, so on fresh records it abstains everywhere and the unchanged agent runs; that is the designed behavior and is reported as such. The compiler, with one frozen candidate, re-derived a byte-identical three-read program, guard, and gate model from the fresh discovery traces (131/132 exact), differing from the retained artifact only in its snapshot pin: a determinism result, not a transfer. The re-derived artifact was then dispatched on all 132 calibration groups and the unchanged continuation run once and graded on the exact task contract: 132 dispatched, 0 misses. Under a single pre-registered rule ($\gamma = \delta = 0.10$, $m = 1$) the one-sided bound on the *end-to-end* violation rate is 0.0173, below $\alpha = 0.05$: the first certificate here whose event includes the continuation. The single rule was adopted prospectively after Table 5 showed the grid buys only its Bonferroni factor; under the primary grid the same count gives 0.035 (0.040 at $m = 2$), and treating the 100 distinct creation days or 85 distinct authors as units gives 0.0228 and 0.0267 under the single rule, so admission does not depend on the rule change. On the 60 fresh test records all four conditions reach 60/60 (20/20 in each class); the re-derived artifact dispatches on every record and cuts requests 75.0%, tokens 82.7%, latency 80.0% and cost 76.6% against the unchanged agent, with no compiled-only miss; the hand-written program ties it. The claim is per-candidate, for one family, conditional on i.i.d. groups from one repository's post-cutoff records; spend was under one dollar.

Continuation-graded drift ablation: an adverse result. The third drift study (`paper/supplementary/drift-continuation-graded-protocol.md`; pre-registered and run 2026-10-07, 1,080 live cells, under one dollar) grades what the two provider-free studies could not: the model's continuation when the evidence it receives is corrupted. On the 60 primary PR-outcome and backlog records, each of the nine perturbations was applied to the tool layer, two arms ran on identical corrupted tools (the retained artifact with its induced verifier, and the same artifact with a permissive verifier; abstention falls back to the unchanged agent on the same corrupted tools), and the final answer was graded against the unperturbed record. The pre-declared adverse branch obtained: the

Table 26: Recurrence-only replay on issue-type routing: per-record means over the 30 sealed held-out records. The first three rows are the retained arms; the last replays the three reads the compiler refused (`limit` is 100 in 130 of 132 traces and 1 in two, and has no witness) with every barrier disabled and one provider request. All four arms are 30/30 exact (McNemar $p = 1$ against each comparator); the replay is the cheapest arm, which is the price of the refusal on this cohort.

Arm	Exact	Requests	Tokens	Latency (s)	Cost (¢)
Unchanged agent	30/30	4.00	4,259	6.16	0.087
Compiled two-read prefix (GAC)	30/30	2.00	2,576	2.97	0.059
Hand-written bundle	30/30	2.00	1,781	3.25	0.055
Recurrence-only three-read replay	30/30	1.00	1,167	2.67	0.038

guarded arm produced a silent wrong answer on 39 of 60 records as graded, all under emptied lists, and the permissive arm produced exactly the same 39 (no discordant record); a 40th record (PR 6694) answered "none" against a real comment but passed the grader's substring test because that comment contains "nonetheless", a grader defect since fixed in the family graders (an audit of every retained graded cohort found no other held-out row affected; the 18-record natural-order study keeps its registered grader, under which one discovery row, issue 1741, is the only row the two rules grade differently). An empty discussion or assignee list lies inside the hull the verifier learned, so it accepts, the artifact dispatches, and the model answers faithfully from the emptied evidence ("no comment evidence"; backlog route flipped), which is wrong against the real record. Where the corruption left the hull, dispatch fell back in both arms before any continuation ran: on renamed keys the interpreter's binding failed in both arms; on nulled fields and error payloads the induced verifier's non-null clauses refused in the guarded arm and the composite's typed projection refused in the permissive arm, and the fallback agent, fed the same corrupted tools, then answered wrong on 30/30 in both arms (fallback misses, outside the 39). Where lists were duplicated or padded the verifier abstained on 11–14 of 30 records whose permissive counterparts answered correctly, and one such fallback (backlog record 5971) itself missed comment grounding (guarded fallback rate on invariant families 0.1389; padded lists are expected-either and outside that denominator). The induced contract therefore adds nothing the typed projection does not already catch on out-of-hull corruption, and catches nothing that lands on a legitimate value; on these records the verifier prevented no silent wrong answer and bought abstention, and one fallback miss, only. This is a statement about this contract on these corruptions with the calibrated model, not about production safety; it is reported ahead of every other reading because the protocol said it would be.

A second model family on the same cohorts. Every headline number comes from one model, so a pre-registered transfer (`paper/supplementary/second-model-replication-protocol.md`) reran the three primary families on `gpt-6-luna`, a different model generation at the same efficiency tier, keeping the sealed discovery traces, splits, and 90 held-out records and recompiling each artifact provider-free under a manifest that pins the new model; the recompiled artifacts are identical to the retained ones (same ids, programs, and 92/0 gates). The decision rule fixed in advance puts one finding first: on issue-type routing the compiled arm missed one record the baseline passed (#6532, an excerpt that pluralized one source word), so its preservation hypothesis fails there; a second record (#3859) failed in both arms through line-ending normalization, and backlog record 2648 failed in the baseline only. Every miss is an excerpt-fidelity error by the model on identical evidence, and the retained `gpt-5.6-luna` arms pass all three records. Structure transferred as predicted: requests per record were unchanged in every arm, the artifact dispatched on 30/30 records in each family, and pooled reductions were 66.6% requests, 63.0% tokens, 59.5% latency, and 60.1% cost against 66.6, 63.1, 64.2, and 58.7 on the calibrated model (Table 29). We read the miss as the case the manifest pin exists for: the certificate is conditional on the pinned model, these artifacts were never calibrated on `gpt-6-luna`, and one compiled-only miss in 90 is what an uncertified transfer looks like. Re-discovery on the second model (design A, `paper/supplementary/second-model-rediscovery-protocol.md`) then ran the whole pipeline on `gpt-6-luna`'s own traces over the same sealed records. Issue-type routing reached 128/132 exact discovery traces and PR-outcome 132/132; the compiler admitted the identical artifacts (same ids and programs, 92/0 gates, one and two candidates at calibration as in the primary runs), which dispatched on 30/30 records and passed 30/30 in every arm, with reductions of 50.0/38.9/32.8

Table 27: Mechanism-removal table: every retained hazard, the guard that caught it, and what a recurrence-only replay of the mined region would have done. Each row is backed by a retained result file in the artifact.

Hazard (where observed)	Guard that caught it	Recurrence-only replay	Retained source (`paper/results/`)
Ungrounded argument `issue_get_comments.limit=100`, in 130 of 132 discovery traces (1 in two) (issue-type, held-out #4420 family)	provenance + synthesis (stage 1/4): literal not invariant, no expression in $\mathcal{L}$, `ungroundable_slot`	replays the majority literal; emits the three-read program	`github_natural_replication/results.json compiler.candidates[0]`
Clean tool replay, wrong downstream answer (Markdown URL dropped, issue 6602) (natural-order study, 17/18 → 18/18 after checked rendering)	continuation contract (post-model)	accepts the answer	`github_natural_live/continuation_replay.json`
Getter-named method that mutates state on 36/36 calls (BFCL executed gold plans)	signed effect declaration (stage 2)	compiles a write into the region	`external_benchmarks/bfcl_compiler_execution.json`
API-documentation prologue before the hot prefix (AppWorld released runs, 4,679/4,680 ReAct and plan-and-execute)	position invariant, Eq. (2), at dispatch	dispatches out of position	`external_benchmarks/appworld_dispatch_preflight.json`
Suffix region dispatched at entry: reordered and duplicated calls (archived pilot, 16.7% pass)	position invariant (compile-time `prefix_only`)	ships the fault	`github_live/pilot_2026-08-03/`
Result outside the induced hull (`pr.state=open`; 40/120 core records) (cross-repository core protocol)	induced verifier V; clean fallback	answers from an unseen class unchecked	`github_multirepo_pr_outcome_core/results.json dispatch.reasons`
Projection type mismatch (`merged_at`, record 7199; 13 events) (PR-outcome pilot)	composite projection verifier; fallback	emits a malformed evidence object	`pr_outcome/pilot_v1/failed_attempt_projection_2026-08-05.json`
Recurrent but under-supported families ($n_{\max}$ 26, 8, 15 < 92) (NESTFUL, API-Bank, BFCL)	stage-6 exact gate	deploys on recurrence alone	`nestful/results.json;api_bank_execution.json;bfcl_compiler_execution.json`
Skewed calibration pool (84/30/2 classes) (`pytorch/pytorch`, core protocol)	stage-6 exact gate ($U = 1.0$)	deploys	`github_multirepo_pr_outcome_core/results.json repository_failures[0]`

and 75.0/80.7/76.0 percent in requests, tokens, and cost; the transfer miss on record 6532 did not recur. Backlog-attention routing refused at compile time: 113 of its 132 discovery traces were exact, below the 116 the 16/8/92 split needs, because on 19 records the model skipped the discussion read, took a two-read route, and left its excerpt ungrounded; no held-out arm was run and no artifact is claimed. The exact-trace filter removed exactly those route-deviating groups. Their inclusion need not produce violations: replay can instead abstain, as the BIRD audit demonstrates. That counterfactual was not executed. The certificate inherits the exact-trace selection even on the primary cohorts, where 130, 131, and 132 of 132 traces are exact; it does not certify the unfiltered population. The paper therefore claims one calibrated model family, reports a second on which the pipeline re-derived two of three artifacts and refused the third, and makes no cross-model certificate claim.

A second provider on the same cohorts. The same design ran on Anthropic's `claude-sonnet-5` through the official SDK, with the unchanged Agents-SDK harness behind a provider adapter (`paper/supplementary/second-provider-replication-protocol.md`): adaptive thinking at effort `low`, parallel tool calls disabled, and the same prompts, tools, structured answers, grader, and compiler settings. Discovery reached 126, 128, and 116 exact traces of 132 (12 of backlog's 16 non-exact traces skipped the discussion read, as on `gpt-6-luna`). Issue-type and PR-outcome admitted the identical artifacts (same ids and programs, 92/0 gates, one and two candidates at calibration) and dispatched on 30/30 records; backlog met the split exactly and both candidates retired at calibration because only 91 and 90 groups were accepted at any threshold, leaving the zero-violation bound at 0.0503 and 0.0509, above α: the floor of Fig. 4 binding on a second provider. The decision rule reports the compiled-only misses first: two on issue-type (records 4248 and 6829) and two on PR-outcome (5401 and 6988). Every miss on this provider, in every arm, is an excerpt that drops Markdown link markup or normalizes whitespace, the issue-6602 mechanism, and the baseline itself missed one and three records. Exact contracts were 29/28/28 of 30 on issue-type (one hand-written episode timed out twice and counts as a failure; the issue-type contract is factuality-only by registration, and two compiled episodes, records 6150 and 5514, also issued one redundant read after the compiled region, which the tool-order check flags and the contract does not count) and 27/27/29 on

Table 28: Second provider, design A: Anthropic `claude-sonnet-5` on the sealed cohorts through the unchanged pipeline. Reductions are against the same-provider baseline; the compiled-only misses are the finding the protocol reports first, and the backlog refusal is the 92-group floor binding.

	Exact held-out contract			Reduction (%)		
Family	Base	Compiled	Manual	Requests	Tokens	Cost
Issue-type routing	29/30	**28/30**	28/30	48.3	40.4	39.8
PR-outcome audit	27/30	**27/30**	29/30	75.0	84.9	83.4
Backlog-attention routing	RETIRE at calibration (91 and 90 of 92 groups, $U > \alpha$; 116/132 exact traces)					

Table 29: Same-cohort transfer to `gpt-6-luna`: the three primary families rerun with the sealed records, discovery traces, and recompiled artifacts behind Table 1. Reductions are against the same-model baseline. The issue-type compiled-only miss (record 6532) is the finding the transfer protocol reports first; under re-discovery the compiler re-derives the identical issue-type and PR-outcome artifacts and refuses backlog routing on the second model's own traces.

	Exact held-out contract			Reduction (%)		
Family	Base	Compiled	Manual	Requests	Tokens	Cost
Transfer (design B): retained artifacts recompiled under the new pin						
Issue-type routing	29/30	**28/30**	30/30	50.0	39.2	33.0
PR-outcome audit	30/30	**30/30**	30/30	75.0	80.8	76.3
Backlog-attention routing	29/30	**30/30**	30/30	74.8	81.0	75.4
Weighted total ($n = 90$)	88/90	**88/90**	90/90	66.6	63.0	60.1
Re-discovery (design A): the pipeline run on the second model's own traces						
Issue-type routing	30/30	**30/30**	30/30	50.0	38.9	32.8
PR-outcome audit	30/30	**30/30**	30/30	75.0	80.7	76.0
Backlog-attention routing	RETIRE at compile time (113/132 exact discovery traces)					

PR-outcome; requests fell 48.3% and 75.0%, tokens 40.4% and 84.9%, and cost 39.8% and 83.4% (Table 28). The structure transfers across providers; excerpt fidelity is the model's, and the exact contract charges it to whichever arm produced it.

A compression comparator ran, and did not engage. Every headline resource number compares a compiled prefix against an *uncompressed* baseline, because deployed compression middleware reports token reductions of the same order on a comparable workload by an entirely different mechanism (Headroom Labs, 2026). A live Headroom v0.5.18 ablation adds `headroom_only/compiled_headroom` arms to the PR-outcome and backlog-attention families on the existing 30-record held-out cohort (`paper/supplementary/headroom-ablation-protocol.md`). Headroom attempted every eligible JSON tool result and pre-model evidence object and applied *none* of them, saving zero tokens in either family, with all four paired quality contrasts holding 30/30 at exact p=1. This is a null engagement at this payload shape, not a measured composition or a measured floor — it does not establish Headroom's behavior on larger contexts — but it does retire the narrower concern that our reduction is compression under another name: on this workload the two systems visibly do not touch the same bytes. It is a compression comparator only; it does not stand in for a workflow-reuse or unguarded-replay baseline. That rerun also re-executed the unchanged agent on the same 30 backlog records and scored 30/30, so the single baseline miss of Table 1 did not recur.

Where "unchanged baseline" is exact. Pre-emission rejection is exact on both runtime paths; only a staging-owning runner can restore byte-identical model input after an attempt has been emitted (Appendix B). Without one, a post-commit verifier failure is an incident rather than a fallback, which is why the compiled region is confined to the pre-commit prefix.

How the calibration cohort must be sampled. The calibration cohort must be sampled before filtering on whether the baseline happened to exhibit the mined region, with violation labels defined for every dispatch-eligible member; post-selecting only recurrent successes would not satisfy Propo-

sition 1. This is why the AppWorld substrate fixes its calibration share at exactly the 92 groups the bound requires and draws them by seeded shuffle over all clean tasks, rather than taking the tasks in which the mined family happens to have the deepest support. The GitHub cohorts' exact-trace filter is such a selection (Section 4); the BIRD study drops it (Appendix F.4).

Provider-free challenge and signature audit. The historical live runs and subsequent audit are distinct.

Two operational caveats of the reported runs have since been closed provider-free: the three primary artifacts were compiled with the perturbation suite disabled and registry signature verification off, and a recompilation from the sealed discovery checkpoints (`paper/scripts/recompile_with_challenge.py`) reproduces the identical programs, splits, artifact identifiers, and 92/0 gates with the suite enabled through a sandbox over the pinned snapshot; all nine perturbation families (reordering, duplicates, large and empty collections, nulls, schema drift, formatting, tool errors, timeouts) complete on the eight development windows of each artifact with zero wrong answers and zero hard rejects, so each record now carries `perturbations_claimed: true`. The same recompilation registers each artifact under an HMAC-SHA256 signing key, reloads it with verification on, and confirms that wrong-key, tampered, and the original unsigned registries are refused (`paper/results/iclr_revision/recompile_with_challenge.json`); no reported number changes.

H Prospective Gate-Frontier Protocol

The retained protocol (`paper/supplementary/prospective-gate-frontier-protocol.md`) fixes candidate selection, splits, thresholds, endpoints, and stopping before acquisition. It targets at least 300 paired cases across at least three repositories, comparing the unchanged agent, learned gate, and support-only gate. A positive result requires at least three nonzero coverage levels and better risk at matched coverage; adverse dispatches take reporting priority. Neither gate showing graded coverage is a predeclared negative outcome. Execution reached four of five repositories, as detailed below; the original protocol remains the source for the complete design.

The 90-record pilot comparator is invalid for a compiled-effect claim (`gate-frontier-pilot-protocol.md`, correction dated 2026-10-08). Its named compiled arm used a `compiled` policy manifest, while the retained artifact was registered under `base`; exact registry resolution therefore found no artifact. The pilot retained factual grades but no dispatch telemetry. Its reported Markdown-link contrast is a difference between stochastic model/tool runs and is withdrawn as a compiled-versus-unchanged result. The independently recorded primary-study excerpt error on issue 6602 still motivates the link stratum; its predictive value is unestablished.

An amended end-to-end issue gate study also retires (`paper/supplementary/graded-issue-end-to-end-gate-protocol.md`). A pinned 474-issue cohort reserved 200 development, 184 calibration, and 90 untouched test issues, enriched for link-bearing comments. The first attempt stopped after 288 ordinary fallbacks because the historical artifact's manifest did not match the current runtime; its checkpoint and estimated cost are retained. A compatible candidate was then re-derived from 80 disjoint historical discovery issues before any new-cohort compiled result. In the amended run, the candidate dispatched on all 200 development and 184 calibration issues, with 4 and 2 final factual-task errors respectively. The frozen eleven-point grid admitted no threshold at $\alpha = 0.05$ and $\delta = 0.10$: at $\eta = 0.02$, 64/184 groups had no errors but $U = 0.0708$; at $\eta = 0.50$, 153/184 had two errors and $U = 0.0546$. The learned gate therefore retired before the 90-issue test. The support-only ablation accepts 184/184 with two observed errors, but it disables the risk budget; no held-out quality or savings comparison follows from this study.

The comparator feasibility bar has also been checked (spike memo in the same directory): the retained feasibility memo did not identify a confirmed public AWO implementation (Abuzakuk et al., 2026) at that check. This historical no-go does not establish present unavailability or eliminate the comparison gap.

Table 30: Disaggregated outcomes for the gate-frontier study. Exact contracts are reported per arm under intention-to-treat; † one support-only episode (record 6708) raised the 120 s provider timeout and was not re-attempted, while the record passed in the other two arms. "Dispatched" counts held-out records on which the compiled program ran; the remainder fell back to the unchanged agent on the guard clause `range:pr.state`. "Coverage in sweep" lists every calibration coverage the frozen grid produces for the learned gate. The support-only arm ($\alpha = 1$) reproduces the same n and k at every η but, because $\alpha = 1$ appends $\eta = 1.0$ to the grid, its Bonferroni split is $\delta/12$ ($U = 0.0507$ at $n = 92$) and it deploys threshold 1.0 rather than 0.5.

Repository	Held-out	Exact (B/L/S)	Dispatched	Coverage in sweep	Req. ↓
`huggingface/datasets`	60	60/60/60	40/60	0.1087 ($U = 0.375$, rejected), 1.0	44.4%
`pandas-dev/pandas`	60	60/60/60	40/60	1.0	44.4%
`psf/requests`	60	60/60/59†	40/60	1.0	44.4%
`pytorch/pytorch`	60 sealed	RETIRE	—	none ($U = 1$ at every η)	—
`streamlit/streamlit`	60	60/60/60	40/60	1.0	44.4%

H.1 Observed results

The sealed cohort targets 300 pooled held-out pairs across the five repositories Section 5.2 already uses, 116 discovery and 60 held-out cases each under frozen-candidate selection. Discovery reaches 580/580 exact traces. Four repositories admit an artifact; `pytorch/pytorch` again retires at compile time. This cohort is drawn from the same frozen snapshots with the same selection seed as the core study and excludes none of its records, so it shares 561/580 discovery, 442/460 calibration, and 130/150 held-out records with the core cohort; at twice the core scale it is a larger sealed re-execution, not an independent replication, and the `pytorch/pytorch` retirement is a near-replay of the core one on a 93–95% identical split. The pooled result is therefore 240 of the pre-registered 300 pairs, short of target for the stated reason.

Accounting. 240 records × 3 arms = 720 attempted episodes, 719 completed. Exact task contracts under intention-to-treat: unchanged agent 240/240, learned gate 240/240, support-only 239/240. The incomplete episode is `psf/requests` record 6708 under the support-only arm, which raised the 120 s provider timeout; the same record completed and passed in the other two arms. The protocol fixed no retry rule, so the episode was not re-attempted and counts as a failure; among the 239 records with three completed arms every arm passes 239/239. Two provenance facts belong here. The `psf/requests` block was executed once, on 2026-08-22T18:59Z, as the single-repository pipeline check under the sealed selection; the five-repository run reused it through the driver's resume path rather than re-executing it. Retained spend for successful attempts sums to $0.41; the two failed `streamlit/streamlit` discovery attempts are not retained.

Dispatch. Calibration coverage is 1.0 in every admitting repository, but held-out dispatch is not: the compiled program ran on 160/240 records (learned gate) and 159/239 (support-only). The other 80 — every open-class pull request, 20 per repository — abstained on the induced verifier's `pr.state` hull, which the discovery pools (0–3 open records each) had fixed at {`closed`}, and ran the unchanged agent. Relative to baseline the learned gate reduces provider requests 44.4%, total tokens 52.3%, observed wall latency 51.0%, and estimated cost 48.3%; the support-only gate — $\alpha = 1$ in an otherwise identical second `compile_grc` pass — 44.4%, 52.4%, 47.9%, and 48.8%. Both arms admit the same artifact; they differ in α, in the grid ($\alpha = 1$ appends $\eta = 1.0$, so the split is $\delta/12$ and $U = 0.0507$ at $n = 92$), and in the deployed threshold (1.0 versus 0.5).

Coverage levels. Table 30 lists every coverage the frozen grid produces. Three admitting repositories accept no calibration group below $\eta = 0.11$ and all 92 at $\eta \geq 0.11$; `huggingface/datasets` additionally accepts 10/92 at $\eta \leq 0.08$, whose exact upper bound 0.375 exceeds α. No repository deploys at an intermediate coverage — and none could: with $|\mathcal{K}| = 92$ the registered bound certifies only $n_\eta = 92$, so the pre-registered criterion of three admissible nonzero coverage levels was unattainable at this cohort size whatever q ranks, and the protocol should have said so. Held-out q is moreover constant within each repository (0.0169; 0.0815), because the entry state is a single record number and every entry-observable feature is a function of it. Per the decision rule fixed above in this appendix, this is the fourth listed outcome: neither gate produces graded coverage. We report it as a null about the design, not as evidence about q: the study audits support-threshold behavior

at full coverage under two risk budgets and does not test the ranking quality of q. The protocol that could (`paper/supplementary/graded-frontier-protocol.md`) needs $|\mathcal{K}| \geq \lceil 92/c \rceil$ groups per repository and an entry state with informative features; it is written and not run.

I Safeguard Ablations with Live Recovery

These follow-ups ask whether compiled prefixes save model work on unused records, whether individual safeguards change deterministic outcomes, and whether those differences survive a live agent's recovery. Cohorts and endpoints remain separate from the six studies' 810 compiled episodes in Section 7; none changes that historical denominator or supplies a new end-to-end risk certificate.

Fresh-record comparison. On 120 previously unused issues from the same historical snapshot, four live arms complete all 480 episodes. Baseline, GAC, and support-only each pass the registered four-field factuality endpoint on 120/120 records; recurrence-only passes 118/120. On the supplementary seven-field endpoint the respective counts are 119, 120, 119, and 118. GAC halves model requests (4 to 2 per episode) and reduces estimated model cost by 45.8% against baseline. Its dispatch decisions are identical to support-only: 119 dispatches and one fallback, so separate stochastic continuations cannot establish a gate effect. The two recurrence-only misses are strict formatting/case differences, not demonstrated semantic false assertions. Prefix length and context also differ, preventing an isolated safeguard attribution. The supplementary conservative 95% interval for GAC minus baseline is $[-3.59, 3.59]$ points on the registered endpoint; equivalence and noninferiority are not established. The previously admitted artifact is fixed, without new discovery or calibration; these are not time-forward records (`experiments/fresh_safeguards/analysis/`).

Deterministic mechanism diagnostic. A disjoint cohort supplies 24 discovery, 96 development, 192 calibration, and 96 held-out issue groups. The study specifies a three-comment prefetch, a requested-limit entry hull, and a response-completeness check; production helpers fit argument bindings and the gate, but end-to-end mining of these contracts is not tested. Each held-out issue receives requested limits 1, 3, and 5 under ordinary responses and responses capped at two comments. Five arms (full, baseline, and removal of scope, completeness, or gate) yield 2,880 provider-free evaluations. Under ordinary responses, full is correct on 288/288 cases versus 260/288 without scope admission; under the cap, full is 288/288 versus 252/288 without completeness checking. The baseline reference policy is correct throughout. The verifier compensates for scope removal under the cap, where that ablation remains 288/288. The fitted gate score is constant (0.00247); all eleven thresholds admit the same 192 calibration groups, and removing it changes no held-out decision. These are deterministic outcomes, not live accuracy or evidence that recovery is impossible (`experiments/safeguard_mechanisms/`).

Stopped pilot and separately sealed completion. The initial 24-issue live pilot attempted 153 of 576 scheduled episodes: 152 completed, 151 were correct, one raised an execution error, and 423 remained unattempted. A separate diagnostic returned the correct answer but reproduced an accounting failure: the old validator rejected reported cache-write tokens. The original failure retained no usage, so this does not identify its exact cause. The completion study prices cache writes at the existing frozen rate and preserves partial usage and unknown-cost reserves. It retains the pilot's prompt, model, tools, answer contract, prefetch, guards, source, timeout, and retry policy; its new protocol and schedule were sealed before execution. Pilot and diagnostic outcomes are excluded from the completion comparison.

Live design and statistical unit. The completion uses all 96 offline held-out issues: 72 *confirmation* groups untouched by the live pilot or diagnostic, and 24 *replication* groups selected for the pilot, including issues it never reached. Every issue has prior offline exposure; replication is not independent of the pilot. Three limits × two response conditions × four arms produce 24 episodes per issue, or 2,304 overall. Full, no-scope-guard, and no-completeness-check differ by one intervention; baseline receives no prefetch. Every arm has the same live continuation and recovery tools, including original call arguments and completeness information. Tool reads use the pinned local snapshot, not a remote GitHub service. The constant gate is retained without a redundant fifth live arm.

Table 31: Live completion: correct answers / scheduled episodes, including failures. No scope removes the requested-limit hull; no completeness removes only that check. All four timeouts occur in confirmation: one each in ordinary baseline, full, and no scope, plus one in capped full.

Cohort	Responses	Baseline	Full	No scope	No completeness
Confirmation	Ordinary	204/216	205/216	186/216	210/216
Confirmation	Capped	211/216	205/216	207/216	204/216
Replication	Ordinary	71/72	70/72	67/72	70/72
Replication	Capped	70/72	68/72	69/72	69/72

Table 32: Confirmation contrasts: full minus comparator, in percentage points. Discordances count full-only / comparator-only correct cases. Intervals resample 72 issue groups, not 216 independent requests, and are descriptive/unadjusted.

Responses	Comparator	Discordances	Difference	95% interval
Ordinary	No scope	24 / 5	+8.80	[3.70, 13.89]
Ordinary	No completeness	4 / 9	−2.31	[−5.56, 0.93]
Ordinary	Baseline	11 / 10	+0.46	[−3.70, 4.63]
Capped	No scope	6 / 8	−0.93	[−4.17, 2.31]
Capped	No completeness	8 / 7	+0.46	[−3.70, 4.63]
Capped	Baseline	4 / 10	−2.78	[−6.48, 0.46]

The model is `gpt-5.6-luna`, with low reasoning and verbosity, `store=false`, no parallel tool calls, at most 2,048 output tokens per request and eight turns, a 120 s request timeout, and no SDK or transport retries. Four workers execute issue blocks; all 24 arm permutations occur three times in confirmation and once in replication within each condition/limit. All scheduled episodes were attempted, with no replacements: 2,300 completed, 2,186 were correct, 114 completed with incorrect answers, and four timed out. Failures count as unsuccessful scheduled attempts (Table 31).

The prespecified paired analysis averages the three limit variants within each issue, then bootstraps issues 10,000 times (seed 20261018), separately for each cohort and response condition. The 95% percentile intervals in Table 32 are descriptive and unadjusted for multiple contrasts. They assume suitable independence between issue groups; author or repository dependence remains possible. The 2,304 episodes are not independent samples. No noninferiority margin or new live risk certificate is asserted.

Scope protection and recoverability. Under ordinary responses, scope admission improves confirmation success by 8.80 points. Full-only correct cases occur in 20 issue groups and ablation-only correct cases in five. Replication gives +4.17 points with interval $[-1.39, 9.72]$: directionally consistent but not decisive. A post-hoc sensitivity on the 214 ordinary pairs where both arms complete gives 23 full-only versus four no-scope-only correct cases; timeout imbalance does not create the net 19-case advantage, but this does not replace the primary analysis.

Incomplete accepted prefetches are defined by source positions and requested scope before inspecting live answers. Without scope admission, only 5/20 such ordinary confirmation cases trigger more reads and only four finish correctly; in replication, 4/8 read further and finish correctly. Without completeness checking under the cap, all 27 confirmation cases read further and 22 finish correctly; all nine replication cases read further and eight finish correctly. The agent often repairs capped evidence, but frequently treats a page complete for a three-comment call as sufficient for a larger request. Requests requiring more than the two-comment cap number 54 in confirmation and 18 in replication.

Full versus no completeness under ordinary responses has identical initial model inputs: differing accuracy reflects separate stochastic continuations, not a verifier effect. Under capped responses, completeness can compensate for the removed scope guard. Neither the unresolved completeness contrast nor the earlier deterministic gap establishes that every safeguard is necessary. Correct prefetches also leave semantic errors after full dispatch; accepted dispatch is not an end-to-end answer certificate.

Table 33: Complete 24-group replication accounting, baseline → full. Model-cost reductions exclude tool-service and lifecycle costs.

Responses	Model requests	Model cost	Total tools	Correct / 72
Ordinary	2.417 → 1.931	−18.2%	1.417 → 1.597	71 → 70
Capped	2.722 → 2.458	−9.1%	1.722 → 2.125	70 → 68

Resource trade-offs and accounting. Every confirmation contrast involving full has an execution failure; therefore its completed-only resource means are descriptive, and overall saving intervals are withheld. Replication has complete accounting (Table 33). Model requests and estimated model cost fall, while total tools increase and slightly fewer answers are correct. There is no demonstrated quality equivalence or overall system-cost win. All discarded prefetch and continuation reads count. Local snapshot reads, concurrency, caching, frozen rates, and omitted engineering/human costs limit transfer to deployment.

The paired model-cost difference intervals are $[-0.00010541, -0.00005827]$ dollars per episode for ordinary responses and $[-0.00007654, -0.00001820]$ for capped responses. New-run recorded cost, including known partial usage, is \$0.952696; four failed-attempt reserves and the carried pilot/diagnostic accounting give \$7.001762 in estimates plus reserves. These are neither invoices nor bounds on actual billing. All 448,212 cache-write tokens were priced; the prior cache-write rejection did not recur.

Interpretation and provenance. The live evidence supports request-scope admission on this constructed workload; it does not establish a completeness-check accuracy benefit, learned-gate value, or superiority to baseline. It does not test end-to-end contract mining or transfer of this guard benefit across models, natural requests, or time. The broader paper's cross-model and post-cutoff studies concern other questions. Source regrading is consistency checking, not independent human gold attestation. All incorrect answers, failure stages, source expectations, metric totals, and paired intervals are retained in `experiments/live_recovery_completion/analysis.json`; the checkpoint, prospective seal, diagnostic, and protocol remain unchanged. The post-run audit regrades answers, checks tool-output digests, and recomputes costs and outcomes. Table 34 indexes the drivers and preceding studies.

J Claims and Evidence

Table 34 maps the principal empirical claims to retained artifacts. Numerical results and their interpretation boundaries appear in the preceding sections; this index supplies source locations without repeating them. The source manifests and publication manifest bind those locations to specific retained files. The statistical assumptions in Appendices A and C apply throughout.

Table 34: Retained evidence index. Paths beginning with a result directory are relative to `paper/results/`; other paths are repository-relative. Angle brackets and braces denote study-specific expansions.

Claim or study	Retained evidence
Preservation and reductions on the primary families	`github_workflow_families/summary.json` (`overall`, `families[].reductions`); per-record arms in `github_natural_replication/results.json` and `github_workflow_families/pr_outcome/final/results.json`, `github_workflow_families/backlog_attention/final/results.json`.
Discordance	`iclr_revision/paired_statistics.json` (`discordance`); `github_workflow_families/backlog_attention/headroom_ablation/results.json`.
Certificate level: per-candidate versus compiler-wide	`iclr_revision/multiplicity_accounting.json` (`u_at_92`, `n_min`); `iclr_revision/recompiled_candidates_pr_outcome.json`, `iclr_revision/recompiled_candidates_backlog_attention.json`.

Claim or study (continued)	Retained evidence
The step-like gate is a sample-size floor	`src/guarded_agentic_compaction/grc/calibrate.py` (`clopper_pearson_upper`, $k = 0$); `gate_selectivity_analysis.json` (`summary`); `iclr_revision/gate_frontier_reanalysis.json` (`grid_rows`); `paper/iclr/figures/gate_floor.pdf`.
810 compiled episodes, one compiled-only failure	Per-record `results[]` of the six studies' result files; counted by `validate_live_extensions` in `paper/scripts/validate_artifacts.py`.
Cross-repository core protocol (newer records)	`github_multirepo_pr_outcome_core/results.json`; `iclr_revision/gate_frontier_reanalysis.json` (`core`); `iclr_revision/pytorch_core_recompile.json`.
Balanced three-repository rerun	`github_multirepo_pr_outcome_balanced/results.json`; `github_multirepo_pr_outcome_balanced/repos/*/results.json`.
Prospective gate-frontier study	`github_multirepo_gate_frontier/results.json`; `iclr_revision/gate_frontier_reanalysis.json` (`gate_frontier`).
Amended issue gate: actual dispatch and retirement	`graded_issue_gate/preflight.json`; `graded_issue_gate/abort_v1.json`; `graded_issue_gate/candidate_v2/summary.json`; `graded_issue_gate/results_v2.json`; `graded_issue_gate/audit_v2.json`; `paper/supplementary/graded-issue-end-to-end-gate-protocol.md`.
External substrates retire at the exact floor	`nestful/results.json` (`compiler.exact_gate`); `external_benchmarks/api_bank_execution.json`; `external_benchmarks/bfcl_compiler_execution.json`, `external_benchmarks/bfcl_gold_execution.json`.
AppWorld admits one artifact	`external_benchmarks/appworld_compiler_execution.json` (`admission`, `exact_replay`, `effect_audit`); `paper/supplementary/appworld-compiler-protocol.md`.
Admissible is not dispatchable	`external_benchmarks/appworld_dispatch_preflight.json` (`pooled`, `by_agent_family`, `first_call_in_released_trajectories`).
Recurrence-only replay	`recurrence_only_ablation/results.json` (`aggregate`, `per_field_exact`); `paper/supplementary/recurrence-only-ablation-protocol.md`.
`gpt-6-luna` transfer (design B)	`second_model_replication/summary.json`; `second_model_replication/issue_type/results.json`; `github_workflow_families/pr_outcome/gpt6_luna/results.json`, `github_workflow_families/backlog_attention/gpt6_luna/results.json`.
`gpt-6-luna` re-discovery (design A)	`second_model_replication/issue_type_rediscovery/results.json`; `github_workflow_families/pr_outcome/gpt6_luna_rediscovery/results.json`; `github_workflow_families/backlog_attention/gpt6_luna_rediscovery/failure.json`; `paper/supplementary/second-model-rediscovery-protocol.md`.
Second provider: Anthropic `claude-sonnet-5` (design A)	`second_provider_replication/issue_type/results.json`; `github_workflow_families/pr_outcome/anthropic_sonnet5_rediscovery/results.json`; `github_workflow_families/backlog_attention/anthropic_sonnet5_rediscovery/failure.json`; `paper/supplementary/second-provider-replication-protocol.md`.
Multiplicity repair: NO-GO	`multiplicity_repair/preflight.json`; `paper/supplementary/multiplicity-repair-protocol.md`.
Group independence	`iclr_revision/effective_units.json` (`primary_ranges`).
Continuation check (introduction example)	`github_natural_live/continuation_replay.json` (`summary`).
Extended issue-type held-out	`issue_type_extended_heldout/results.json` (`compiled_only_bound`, `dispatch`); `paper/supplementary/issue-type-extended-heldout-protocol.md`.

Claim or study (continued)	Retained evidence
Read-only prologue	`external_benchmarks/appworld_dispatch_prologue_preflight.json` (`by_agent_family`); `tests/unit/test_dispatch_prologue.py`; `paper/supplementary/read-only-prologue-protocol.md`.
Perturbation challenge and signed registries	`iclr_revision/recompile_with_challenge.json`; `paper/scripts/recompile_with_challenge.py`.
Drift ablation (provider-free, demonstration substrate)	`drift_ablation/results.json` (`pooled`, `decision`); `paper/scripts/drift_ablation_study.py`; `paper/supplementary/drift-robustness-ablation-protocol.md`.
Recorded-replay drift ablation (provider-free, 90 primary records)	`drift_recorded_replay/results.json` (`pooled`, `families.*.applicability`); `paper/scripts/drift_recorded_replay_study.py`; `paper/supplementary/drift-recorded-replay-protocol.md`.
Post-cutoff cohort; end-to-end certificate (PR-outcome)	`time_forward/pr_outcome/certificate.json`, `time_forward/pr_outcome/evaluation.json`, `time_forward/pr_outcome/selection.json`; `paper/scripts/time_forward_pr_outcome_study.py`; `paper/supplementary/time-forward-end-to-end-protocol.md`.
Continuation-graded drift ablation (live, 60 primary records)	`drift_continuation_graded/results.json` (`per_cell`, `decision`); `paper/scripts/drift_continuation_graded_study.py`; `paper/supplementary/drift-continuation-graded-protocol.md`.
BIRD: a live SQL agent on a public benchmark (five databases, two designs)	`bird/summary.json`, `bird/<design>/<database>/compile.json`, `bird/<design>/<database>/evaluation.json`; `paper/scripts/bird_sql_agent_study.py`; `paper/supplementary/bird-sql-agent-protocol.md`.
BIRD extensions: rotated rerun, second model, second provider, train split	`bird/extensions_summary.json`, `bird/{rotated_rerun,replications,train}/`; `paper/scripts/bird_sql_agent_study.py`; `paper/supplementary/bird-sql-agent-extensions-protocol.md`. All-scheduled denominators: `bird/denominator_audit.json`; `paper/scripts/audit_bird_denominators.py`. Paired uncertainty and database/cost sensitivity: `bird/quality_sensitivity.json`; `paper/scripts/bird_quality_sensitivity.py`.
Fresh-record safeguard comparison (480 live episodes)	`experiments/fresh_safeguards/analysis/analysis.json`, `experiments/fresh_safeguards/analysis/results.md`, `experiments/fresh_safeguards/analysis/error_audit.md`; `paper/scripts/fresh_safeguard_comparison.py`.
Deterministic scope/completeness diagnostic	`experiments/safeguard_mechanisms/artifacts/run/summary.json`, `experiments/safeguard_mechanisms/artifacts/run/outcomes.jsonl`, `experiments/safeguard_mechanisms/artifacts/seal.json`; `paper/scripts/safeguard_mechanism_study.py`.
Stopped live pilot and diagnostic	`experiments/live_safeguard_recovery/OBSERVED.md`; `experiments/live_recovery_completion/diagnostic.json`, `experiments/live_recovery_completion/diagnostic_driver.py.txt`. Pilot outcomes are excluded from the completion comparison.
Completed live recovery (2,304 attempts)	`experiments/live_recovery_completion/README.md`, `experiments/live_recovery_completion/seal.json`, `experiments/live_recovery_completion/checkpoint.json`, `experiments/live_recovery_completion/analysis.json`, `experiments/live_recovery_completion/FINDINGS.md`; `paper/scripts/complete_live_recovery.py`, `paper/scripts/recovery_observation.py`.